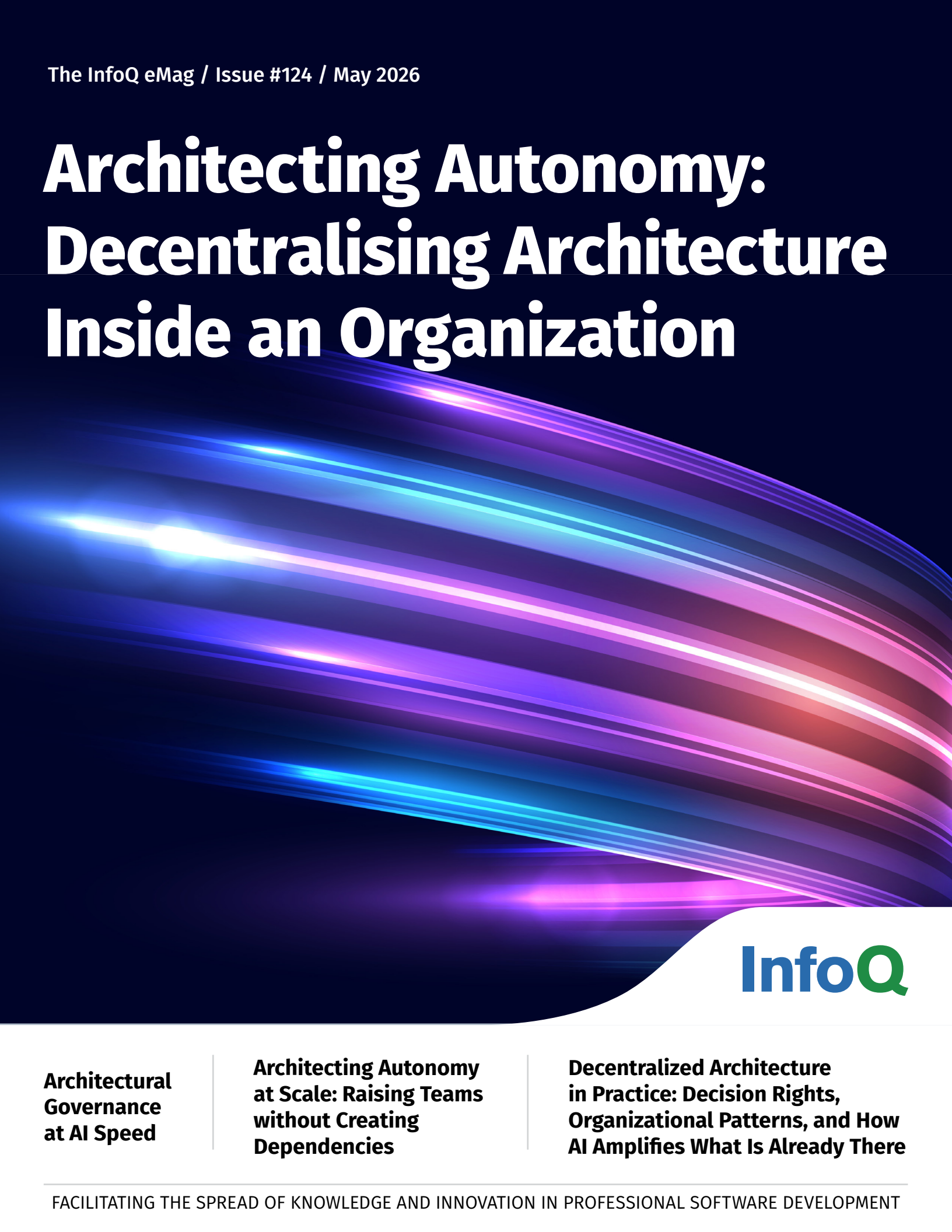


Architecting Autonomy: Decentralising Architecture Inside an Organization



InfoQ

**Architectural
Governance
at AI Speed**

**Architecting Autonomy
at Scale: Raising Teams
without Creating
Dependencies**

**Decentralized Architecture
in Practice: Decision Rights,
Organizational Patterns, and How
AI Amplifies What Is Already There**

Architecting Autonomy: Decentralising Architecture Inside an Organization

IN THIS ISSUE

05

Architectural Governance
at AI Speed

39

Decentralizing Architecture at Scale:
A Lean Value Tree Approach to
Autonomous Teams, Federated
Governance, and Platform Engineering

15

Architecting Autonomy
at Scale: Raising Teams
without Creating
Dependencies

49

Decentralized Architecture
Governance: From Review
Boards to Golden Paths and
Practitioner-Led Standards

24

Decentralized Architecture
in Practice: Decision Rights,
Organizational Patterns, and How
AI Amplifies What Is Already There

55

Decentralized Architecture
Governance: Decision Rights,
Enabling Platforms, and the Shift
from Gatekeeper to Adviser

33

Architecting Autonomy: How
Decentralizing Architectural
Decisions Enables
Organizational Scale

62

Contributors

A LETTER FROM THE EDITOR



Luca Mezzalira

Luca Mezzalira is a Principal Solutions Architect and the author of [Building Micro-Frontends](#). He helps organizations design evolutionary architectures and leads modernization programs for global engineering teams. Luca is also the host of the [Micro-Frontends in the Trenches podcast](#), where he explores the future of distributed frontend systems getting deeper into heuristics, pitfalls, and lessons learned from teams that turned this architectural evolution into a competitive advantage. The podcast can be listened to on Spotify, Apple Podcasts or YouTube.

There is a peculiar irony at the heart of scaling organisations: the larger and more complex a system becomes, the stronger the gravitational pull toward centralised control, and yet that same centralisation is often what slows everything down.

Architecture review boards become queues. Principal engineers become bottlenecks. Teams with genuine context and skin in the game wait for sign-off from people who are three steps removed from the problem. The system optimises for consistency and ends up sacrificing the very adaptability it was trying to protect.

This is not a new tension. Conway's Law has been telling us for decades that our systems will mirror our communication structures, whether we intend them to or not. What is new is the pressure. AI is compressing timelines, accelerating delivery cycles, and lowering the barrier to building at scale. Teams that once needed months to prototype a capability can now move in days. That velocity is an asset,

but only if your architectural governance can keep pace with it. When it cannot, you do not get consistency, you get fragmentation at speed, which is considerably harder to untangle than fragmentation at the old pace.

The instinct to respond by tightening central control is understandable, but it is the wrong move. What organisations actually need is a different model of governance, one that distributes decision-making authority to the teams closest to the problem, while maintaining coherence across the broader sociotechnical system. That distinction matters. Decentralising architecture is not the same as abandoning architecture. It is a deliberate shift from gatekeeping to guardrails, from approval chains to principles, from control at the centre to coherence at the edges.

In practice, this means rethinking what architectural roles actually do, where decision boundaries sit, and what enabling platforms

need to exist so that teams can move autonomously without inadvertently working against each other. It means federated governance models where teams own their architectural decisions within well-understood constraints, rather than submitting to a centralised authority that cannot possibly have full context across every domain. And it means accepting that the architect's most valuable contribution is no longer the decision itself, it is creating the conditions under which good decisions can be made consistently, by others, at scale.

None of this is straightforward. Fragmentation, duplicated effort, and quiet drift away from strategic direction are real failure modes. So are the cultural and leadership shifts required to make

decentralisation stick, because this is fundamentally an organisational transformation, not a technical one.

What follows is a collection of perspectives from practitioners who are working through exactly these challenges. You will find frameworks for redistributing architectural authority without losing strategic direction and patterns for federated governance that scale across large engineering organisations. The thread connecting all of it is the same question: how do you build a system that is coherent enough to function as a whole, and autonomous enough to evolve at the edges, without one property destroying the other?



Architectural Governance at AI Speed

Kyle Howard, Senior Backend Engineer

Christian Johansen, Independent Software Architect and Founder @GalaxiTech

Dana Katzenelson, Staff Backend Software Engineer

Brian Rhoten, Solution Architect for Public Safety

Warren Gray, Technology Leader

Code is Now a Commodity, Alignment is Still Not

GenAI has slashed the effort required to produce code, and rapid prototyping is increasingly common. As a result, the software development lifecycle is now constrained by an organization's ability to bring ideas into alignment and maintain cohesion across the system.

Historically, organizations have relied on manual processes and human oversight to achieve architectural cohesion. Startups rely on key individuals to catch misalignment between architectural intent and implementation. Enterprise-level organizations attempt to maintain cohesion through change boards and proliferating ADRs and documentation. In both contexts, identifying

misalignment is slow because it requires synchronous dependence on a central authority. In the startup case, development teams are stuck waiting for busy experts. In the enterprise case, they have to wait on review boards and sift through documented guidance with the hope that what they find has not become obsolete. GenAI exacerbates this by accelerating the production of work that's subject to review. Where previously only developers were producing code over days or weeks, executives and product managers can now vibe-code functional prototypes in minutes or hours. As a result, development teams are left with an impossible choice: be beholden to the pace of manual oversight at the cost of velocity, or push forward without knowing whether they are aligned.

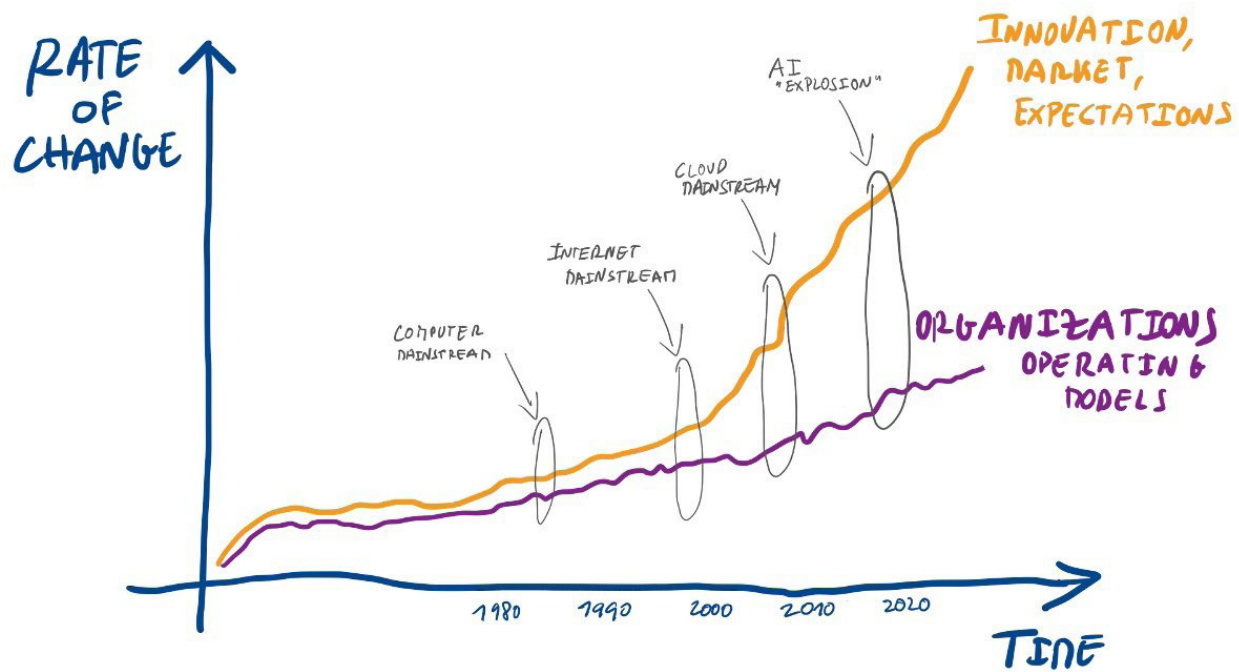


Fig. 1. From [Eduardo Da Silva](#), used with permission

Over time, these small pushes compound into architectural fragmentation, which the organization responds to with more process and stricter guidelines, which further increase the difficulty of releasing software in alignment. This is a vicious cycle that slows delivery and blunts innovation.

Declarative Architecture, Decentralized Alignment

Scaling alignment in the GenAI era requires that organizations move beyond manual oversight toward automated guardrails that enable teams to make safe decisions autonomously. To this end, we propose a strategy of **declarative architecture** to scale architectural governance.

Declarative architecture is the practice of distilling architectural decisions and constraints into machine-enforceable declarations of intent that enable safe independent action. Each declaration governs a bounded scope: a clearly defined context within which it has authority. Without that boundary, declarations become the same sprawling guidance they were meant to replace.

Declarative architecture is not about making better decisions. Instead, it focuses on making decisions impossible to ignore. Instead of tracking down and interpreting architectural documents or waiting on experts and review boards, a machine-readable declaration of intent makes the conformant path the path of least resistance. Validation of compliance stops being a function of awareness and memory, and is instead encoded into tools that meet developers where they already are: in their editors, their pipelines, their code review tools. Machine-readability is not a technical nicety in this framework: it is the core requirement. A declaration that can only be understood by a human still depends on that human being in the loop. Only a declaration that machines can reason over can scale governance beyond the bandwidth of any individual or review board. In the following sections, we examine practical applications of declarative architecture distilled from event models, OpenAPI specifications, and ADRs.

Credit Adam Dymitruk Adaptech Group

An Event Model describes how information flows through a system, and is typically produced through collaborative modeling exercise.

backed by a formal schema. This eventmodel.json forms the declared architecture of the system: a machine-readable map of architectural intent from which aligned implementations can be generated and against which agents can evaluate implementation.

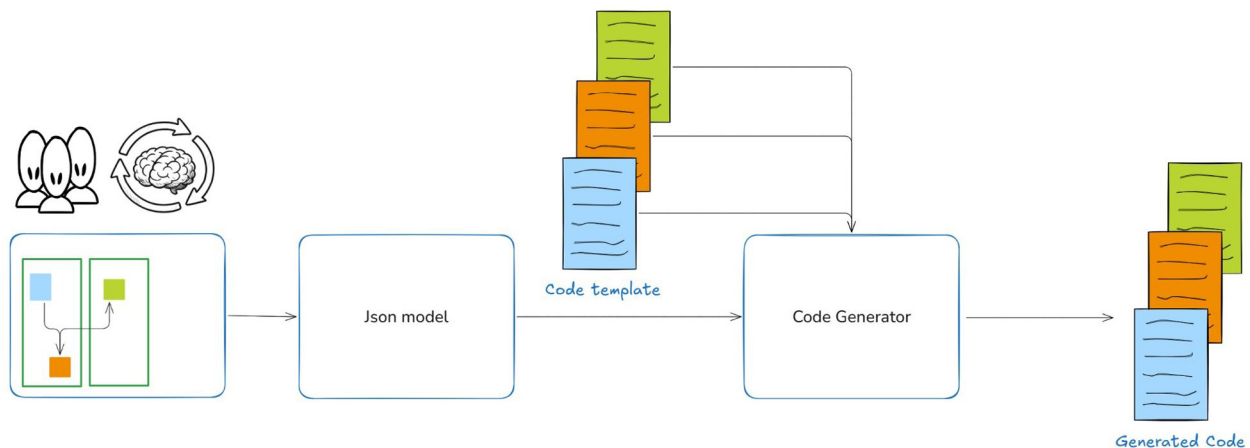


Fig 3. From event model to generated code

1. Powering Automation

Vertical slices are bounded in scope and contain the declarations needed to describe a single unit of behavior. This minimal scope is what unlocks automation at every level. Code artifacts can be deterministically generated from the Event Model using code templates – no AI required. And because each slice is self-contained and code artifacts are also organised by slice (see [vertical slice architecture](#)), if something is wrong, you replace one slice rather than untangle a web of dependencies. (Contracts towards external dependencies still need to be honored, though). For what templates can't cover, the formal schema enables AI to go further. Martin Dilger from Nebulit has defined a [Claude rule file](#) that analyses a code base and generates an Event Model from it. Domain experts can then review and refine the model through an AI Agent, enabling conversational programming at a higher abstraction level than code. Because the scope is minimal, both humans and AI can focus on one slice in isolation, dramatically reducing cognitive load. This same minimal scope is what makes the Ralph Loop (explained below) possible, turning each slice into an independent iteration target for an AI agent.

2. Decentralizing Architectural Alignment at Scale

Event Models enable decentralization, but only if the modeling itself is collaborative across team boundaries. If teams model independently in silos, each team's Event Model may be internally coherent while the product as a whole fragments. Organizations like [Adaptech Group](#) and [Nebulit](#) have

practiced multi-domain collaborative modeling covering many teams, demonstrating that this scales beyond a single team boundary. Technical alignment mechanisms like `architecture.md` and OpenAPI validators (mentioned below) enforce how things are built, but cannot ensure teams are building the right thing together. That requires shared modeling on a broader scope, like Event Modeling or other [Collaborative Design techniques](#). The good news: technical governance frees human attention from policing implementation details, creating space for product-level alignment.

3. The Ralph Loop: From Slices to Shipping

The [Ralph Wiggum Loop](#) is one of several AI-looping techniques where an agent repeatedly attempts a task until completion criteria are met, restarting with fresh context each iteration to avoid degraded reasoning. In early iterations, using an [architect in the loop](#) is common for tuning and validating the setup. [Here](#) is an example showing the result of the second iteration of an [AI Assisted ADDR process](#).

Each vertical slice from the Event Model maps to one iteration target: the agent reads the slice spec, implements, validates against Given-When-Then specifications, and loops until they pass. Because each slice is minimal scope, it fits in a single context window. Even cheap models produce reliable results under these conditions.

Spec + Vertical Slice = Candy for AI (Credit Martin Dilger)

```
ralph-loop/
├── decisions.md           # cross-plan: architectural decisions (ADR-like)
├── learnings.md          # cross-plan: distilled patterns and insights
├── AGENTS.md             # agent behavior file (updated by human review)
├── plan-user-registration/
│   ├── state.md          # cursor for this plan
│   ├── implementation-plan.md
│   └── logs.md
├── plan-order-processing/
│   ├── state.md          # cursor for this plan
│   ├── implementation-plan.md
│   └── logs.md
```

Fig. 4. Ralph Loop file structure

The structure tells the story: plan folders are independent and parallelizable. Each has its own progress marker, implementation checkboxes, and logs. Feedback loops produce decisions and learnings, which accumulate across plans. When these are significant enough, a human developer reviews them and updates AGENTS.md. The agent's behavior evolves based on curated experience, not automated self-modification. Every iteration generates organizational knowledge as a byproduct of doing the work. [\[Link to implementation repository\]](#)

Section 4: The Prospect for Event Modeling

The below Wardley Map illustrates the industrialization of AI, combined with maturing Event Model tooling and an emerging schema standard, resulting in something novel: Conversational AI Event Modeling. This process brings together human domain experts and GenAI to model systems through conversation rather than code. The implications are worth pausing on. Within the Event Modeling workflow described above, code becomes disposable. Slices are minimal scope, fully specified, governed by templates and architectural

constraints. You don't maintain a slice. You regenerate it from the spec. This reframes legacy modernization: instead of rewriting millions of lines, you model the domain and generate fresh code from it, ensuring continuous alignment.

Event Modeling operates at the level of domain behavior, distilling business intent into machine-enforceable slices. But the same declarative pattern applies beyond the domain boundary. Once behavior is modeled and generated internally, external interfaces must also remain coherent. Declarative architecture does not stop at slices – it extends to the contracts that bind systems together.

OpenAPI Validators as Declarative Architecture

Consider a mid-sized enterprise running a highly distributed SaaS platform composed of hundreds of components owned by dozens of autonomous teams, connected primarily through HTTP APIs. Historically, this organization has relied heavily on implicit, single-person architectural ownership and “standards and practices” documentation. The “correct” approach depends heavily on which people a team chooses to consult and which documents

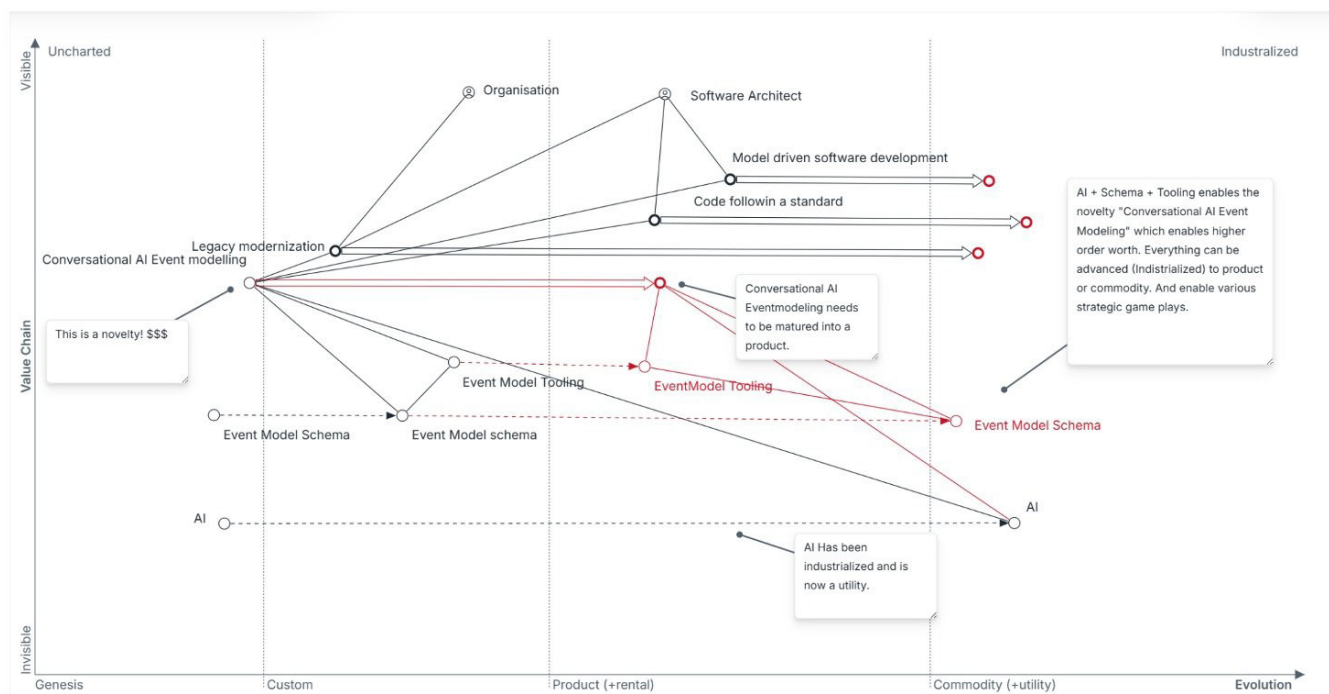


Fig. 5. Wardley Map: Event Model Tooling Evolution

they discover first. Teams struggle to locate the most appropriate path forward, increasing the risk of misalignment at project start. This misalignment compounds during execution as ad-hoc decisions are made, exacerbating drift between architectural intent and implementation.

This organization is at a pivotal point in its growth and seeks to launch a customer-facing API surface. The organization is prioritizing compatibility and ease of use to maximize adoption of this new surface. It also wants to maintain the velocity benefits it derives from highly autonomous teams, and it recognizes the bottlenecks associated with its current ad-hoc approach to architectural governance. Finally, the organization wants to ensure that the architecture keeps up with changing business requirements.

To reconcile this tension, the organization is applying a declarative approach to architecture oriented around [OpenAPI](#) specifications, with standards and validating tools (i.e. linters) defined by a centralized platform team. These validation tools encode the architectural intent of the platform team and provide automated feedback to development teams about their conformance to that intent, and are integrated into CI/CD to safeguard the production environment against misalignment. Those same tools are also integrated

into developer workflows (e.g. in the IDE, through GitHub status checks) for fast feedback and to shift-left the detection of misalignment. This model of centralized decision-making and decentralized enforcement ensures consistency for cross-cutting API concerns (e.g. path conventions, versioning and backward-compatibility, pagination, error structure), ultimately resulting in a frictionless developer experience for internal product teams and external API consumers.

This approach fits naturally into the organization's existing software lifecycle, which leans heavily on declarative infrastructure and independent deployability. Each team's OpenAPI specification is used as the deployment configuration for their API – the platform translates OpenAPI documents into API Gateway configuration, authorization policy, and user-facing documentation. Because all development teams share a standard build-test-deploy pipeline, the platform team has a single, predictable point at which to enforce architectural intent – no per-team coordination required. These teams continue to use the existing processes they're familiar with; deploy-time validation precludes the possibility of misalignment. A deployment cannot succeed unless validation passes, so misaligned work can never reach production.

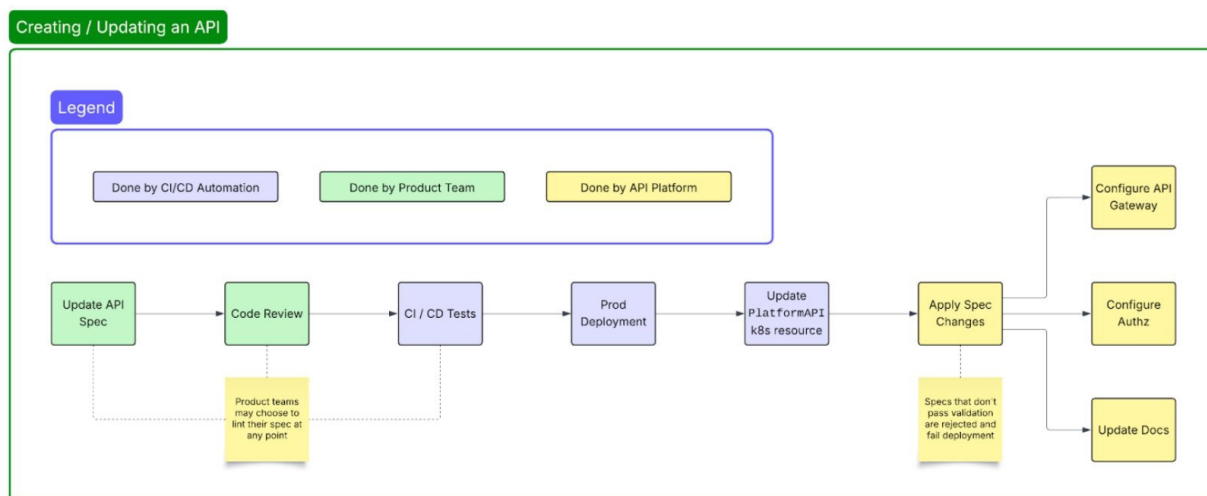


Fig. 6. API Lifecycle with Build-Time and Deploy-Time Validation

This model is designed to produce low cognitive load and high autonomy for dev teams, strong architectural coherence in production, and ultimately a consistent and intuitive API experience for customers. Early signals suggest it is working as intended: initial API teams have used passing validation results as objective evidence to satisfy governance requirements that previously required days or weeks of manual review.

Because architectural intent is encoded in machine-verifiable rules, alignment becomes observable. The platform team collects telemetry from the validators – validation error rates, lead time to change, and product team feedback – to identify systemic friction. Repeated violations often indicate unclear or impractical standards. High lead times may signal excessive constraints. Sustained low violation rates confirm stable conventions.

Just as the R part of the [ADDR](#) process is used to refine API designs, this validator feedback allows the architectural intent itself to evolve. Instead of periodic standards revisions or ad-hoc debates, the platform team refines rules based on observed behaviour. The architecture adapts with the business, and teams adapt with it.

Deterministic validation guards the edges of the system, but architecture is more than syntax and structure. It is accumulated judgment – and judgment does not fit neatly into a linter rule.

On Architectural Decision Records

“One of the hardest things to track during the life of a project is the motivation behind certain decisions.” – Michael Nygard, [2011](#)

[Architectural Decision Records](#) (ADRs) serve as the reasoning behind any significant change within a software development project. They aim to allow the transfer of context, which is often lost due to the constantly shifting nature of agile development, from the decision maker to anyone who subsequently needs to maintain a system.

ADRs provide a framework for decision making focused not only on identifying the tradeoffs and alternatives, but also adding contextual information which often affects a decision more than just choosing the better solution. The ADRs serve as a history of the collective forces that have driven the evolution of a software system.

With the advent of GenAI, however, research time is vastly reduced, and the expectation of productivity seems to be increasing, ADRs might not be allowed to fulfill their purpose effectively. Humans eventually become the bottleneck and this can become a problem, especially in organizations where these decision records need to be approved before implementation. In his 2011 [blog post](#) Michael Nygard also stated that “Nobody ever reads large documents” as a part of his context for developing ADRs. What happens when, because documents can now be easily mass-generated, nobody reads small documents either? Could agents be the solution?

[architecture.md](#) as Declarative Architecture

It is one thing to statically analyze your OpenAPI specs with architectural fitness functions, but in the age of [Spec Driven Development](#), we must go further. A deterministic validator may greenlight syntax, style, and structure, but what about the *intent* of the application?

Deterministic tools only know what you tell them. They cannot call out that a perfectly valid REST endpoint violates a core architectural decision requiring asynchronous events for internal communication without relying on brittle whitelists and exceptions. Historically, we tried to solve this gap by “shifting left” and pointing developers to a massive wiki of Architectural Decision Records (ADRs). But that buried teams under cognitive load; instead of shifting left, we were *dumping left*, guaranteeing that as the system scaled, the architecture would drift.

We need a mechanism that understands the *spirit* of the guidelines as well as the letter. We need to distill

our architecture into an executable format that an agentic model can enforce, mentoring developers in real-time rather than just gatekeeping their code.

Enter `architecture.md`

This file is a rigorous reduction of our architectural guidance into an agent-friendly directive, applied liberally and consistently with every execution. Giving an agent our massive graveyard of wiki articles would only lead to confusion and hallucination, grinding the pipeline to a halt. Instead, every ADR must be boiled down into a brief directive that both human and agent can internalize and apply to the code:

- We do X instead of Y.
- We prefer A to B when C is true.

Each ADR and technical requirement is given its space and treatment, encapsulated into an atomic unit of intent.

Closing the Loop: Automated Governance

A static file is useless if it is allowed to rot. To prevent this `architecture.md` from becoming just another stale wiki page, we treat it as code. A governance agent can run nightly, comparing the repository's manifest against the central ADR corpus. If it detects a new or superseded decision, it opens a pull request to update the spec, ensuring the repository never drifts from the enterprise baseline.

As your implementation matures, you may run into a bloated `architecture.md`, but that's easily overcome by using a directory structure (similar to [this proposal](#) with `agents.md` to keep your scoped sections in separate files.)

With a verified manifest in place, the runtime enforcement becomes simple. Whether running as a GitHub Action or a local IDE Copilot, the agent fetches only the relevant clauses (e.g., loading Data rules only when schema files are touched),

```
# architecture.md
# Executable Architectural Manifest

## Service Communication
* [ADR-088] [Warn] Require asynchronous Kafka events for inter-service communication.
Reject synchronous REST calls unless communicating with external 3rd party vendors.
* [ADR-012] [Block] All public-facing APIs must be defined in OpenAPI 3.1 format before
implementation code is written.

## Data & Persistence
* [ADR-004] [Block] Services must own their data. Direct database connections to another
service's schema are strictly forbidden.
* [ADR-055] [Warn] Prefer DynamoDB for high-throughput, simple-access patterns. Use
Postgres only when complex relational joins are required by the domain.

## Infrastructure & Scaling
* [System-Intent] [Block] All microservices must be designed as stateless execution units
to support spot instance interruption. Session state must be externalized to Redis.
* [InfoSec] [Block] All authentication must utilize the corporate IdP endpoints.
Application specific user tables are forbidden.

## Observability
* [ADR-067] [Warn] Log messages must use structured JSON and include the `correlation_id`
from the request context.
* [ADR-073] [Warn] Do not log request bodies containing PII. Use the `Redact()` helper on
the payload before logging.
```

Fig. 7. Example of what an `architecture.md` might look like

evaluates the semantic intent of the code, and provides in-situ remediation.

If your architecture is an executable manifest, your tools stop being critics and start being collaborators. The same file that guides an agent's code review can be the seed for your generated fitness functions and the validator for your infrastructure's identity. We aren't just shifting left; we're finally giving the "Left" a map.

Conclusion

In the era of GenAI, where drafting code and documentation is a commodity, ensuring consistency and standards across an organization is now more critical than ever. Through the use of declarative architecture and feedback loops, organizations are able to continuously evolve and automate enforcement of architectural intent. By applying industry-standard techniques like Event Modeling, OpenAPI inspection, and Architectural Decision Records and distilling the results into machine-enforceable declarations of architectural intent, organizations enable teams to move quickly and with high-confidence alignment. This pattern can be extended into agentic flows to cover intent

that previously only humans could validate work against.

To eliminate the bottlenecks of centralized governance, architectural decisions must be encoded directly into the tools that generate and validate implementation. The conformant path should be the easiest path. Generators belong at project inception and inside developer workflows. Validators belong in the IDE, in pull requests, and at deployment. Governance that lives outside the delivery pipeline will be bypassed; governance embedded within it becomes invisible and automatic. These mechanisms must be paired with feedback loops that continuously evolve declared architecture.

Code is now abundant. Alignment is not. The future of architectural governance is not more review boards or better documentation. It is declared intent, continuously enforced and continuously refined, operating at the same speed as the systems it governs.

SPONSORED ARTICLE

The Senior Engineer's Missing Pressure Test

Why peer feedback from outside your organization is the check that disappears as you get more senior

The decisions that shape your career rarely get pressure-tested at work. Once you reach a certain seniority, the team defers to you, and leadership trusts your judgment.

When you make a call on a team boundary, on build vs. buy, on which AI platform to commit to, and then you wait years to see if it was right.

That check still exists outside your own company. The [InfoQ Online Certification Programs](#) are 5-week online cohorts that bring senior engineers, architects, and technical leaders together with peers from other companies.

The InfoQ Certification Programs

Luca Mezzalana, Principal Solutions Architect, facilitates the [InfoQ Certified Architecture Program](#). The [InfoQ Certified AI Engineering Program](#) is facilitated by Hien Luu, Sr. Engineering Manager at Zoox and author of *MLOps with Ray*. New topics are added as senior practitioners ask for them.

Each program is built around a curated set of QCon talks. Each week, participants watch a talk in advance, then spend four hours in a live session with the facilitator and their cohort applying the frameworks to real decisions from their own work. The capstone is a technical article that participants draft and refine throughout the program. The strongest pieces are published on InfoQ.com.

The view from outside your company

Participants articulate the trade-offs of a decision they're about to make, then field questions from senior engineers in unrelated industries. As Luca put it: "On the architecture side, we are siloed inside

our company, inside our industry. If you can expand your horizon and see how other people are treating the same problem, you might find something that makes your situation 10x better."

Architecture: the sociotechnical side of the role

The Architecture cohort moves from technical decisions to the organizational work of implementing them. Across five weeks, [the syllabus](#) covers resilience patterns for distributed systems, decentralized decision-making with Architectural Decision Records (ADRs) and Requests for Comments (RFCs), platform engineering as a sociotechnical practice, and AI integration.

AI Engineering: from prototype to production

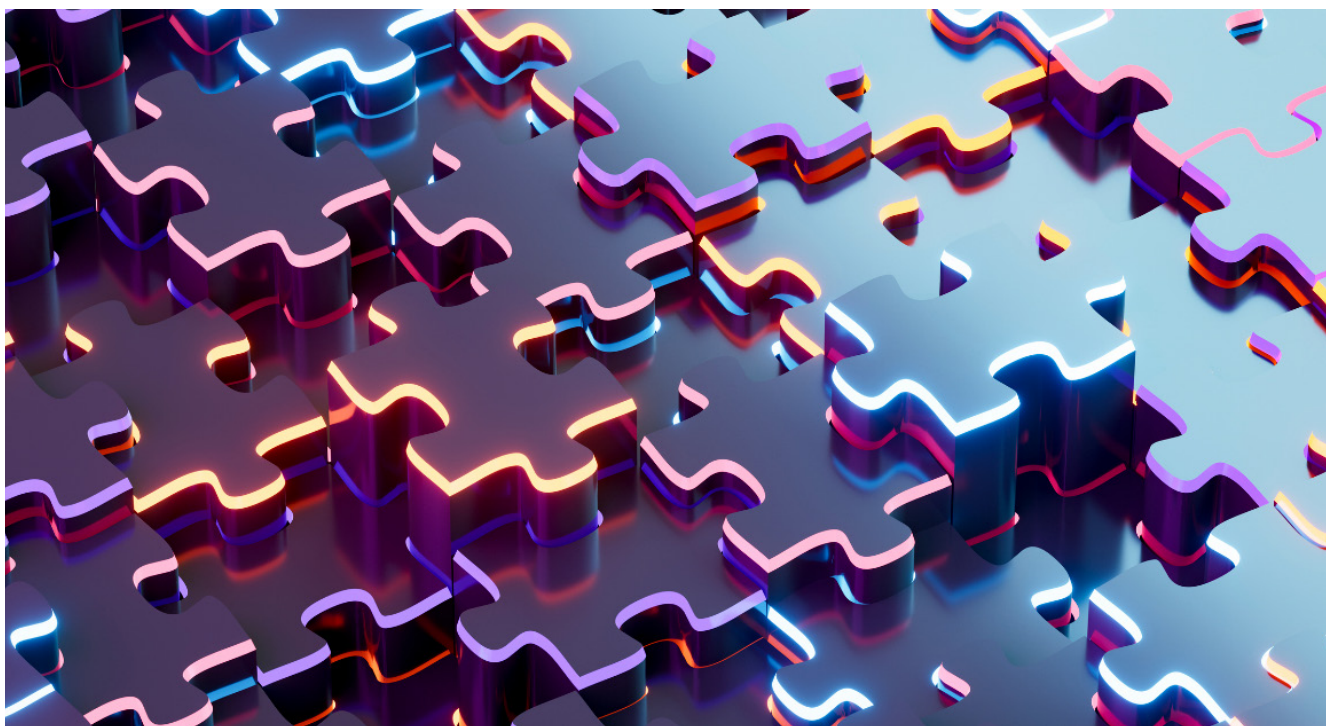
The AI Engineering cohort [covers](#) what comes after the demo: retrieval augmented generation (RAG) and context pipelines, agent design, AI platform and infrastructure decisions, and the evals and reliability disciplines that keep AI systems running in production. The demo works but prod is different, and most teams are making those calls for the first time, with no peer outside the team to pressure-test the response until something breaks.

What participants say

Chinmay Sawaji, Senior Software Engineer at Klaviyo: "The single most valuable outcome has been improving how I write and think. I can now articulate trade-offs in a way that improves my proposals at work."

David Holliday, Product Manager at Munich Re: "This cohort gave me structured time each week to step back and think about what it really means to be an architect."

Learn more about the [InfoQ Online Certification Programs](#).



Architecting Autonomy at Scale: Raising Teams without Creating Dependencies

Shweta Aggarwal, Senior Solution Architect @Liberty IT
Ron Klein, Software Architect

Decentralized decision-making is far from a new paradigm; it has shaped systems in nature, economics, and human societies for centuries. A classic example is the Red Harvester Ant colony - a system that exhibits remarkably sophisticated and efficient behavior without any single ant, not even the queen, directing day-to-day activity.

Each ant operates using limited, local information: the rate of antennal contact, pheromone intensity, and environmental signals. From these simple inputs, complex patterns emerge.

Task distribution, information sharing, and task switching arise organically from local interactions. When a food source proves abundant, ants reinforce the trail with additional pheromones, creating a positive feedback loop that attracts more ants - without any "manager" assigning work. The result is a highly adaptive colony that responds in real time to changing conditions.

If ant colonies can coordinate thousands of agents without a central commander, can large-scale engineering organizations do the same? Or are we

doomed to rely on architectural “ivory towers” to keep chaos at bay?

For years, architecture has been framed as a centralized control function - a necessary authority designed to impose order through top-down decisions.

That model can work in the early days. But as organizations scale, what once provided clarity often becomes friction. The very mechanism meant to prevent chaos turns into the bottleneck that slows progress.

This article presents a **practical framework for decentralizing architecture**, using clear decision boundaries, shared principles, and guardrails instead of gates to enable aligned local decisions while preserving coherence at scale.

The Architecture Bottleneck

Centralized architectural decisions are much like a centralized lock in a distributed system: it works at low scale but becomes a bottleneck as complexity grows. It can be effective in small organizations, during early engineering setup, or in time-bound initiatives that require tight alignment to prove value and set direction.

In mature environments, centralized decision-making can introduce new problems:

- **Slower time to market:** when every significant decision must pass through a central authority, delivery pipelines turn into queues.
- **Context blindness:** central architects may not possess the deep, fast-evolving domain knowledge held by individual teams, leading to suboptimal or misaligned decisions.
- **The ivory tower effect:** when teams perceive architectural decisions as detached from real constraints, trust erodes and friction increases.

As domain complexity grows, centralization becomes harder to sustain. For example, in industries like insurance, *claims, billing, policy administration*, and *underwriting* operate under different constraints and workflows. No central group can maintain deep expertise across all domains.

Decentralization does not mean *anything goes*. It is a progression: from *dependence* on central approval, to *independence* within shared principles and platforms, and ultimately to *interdependence*, where central and domain teams collaborate as peers to shape architectural direction.

The Parental Metaphor: A Framework for Growth

A useful way to frame this is to view architects like parents. Effective parenting is not about controlling every move, but about setting boundaries, cultivating judgment, creating safe spaces to learn, and gradually transferring responsibility.

Growth applies not only to teams, but to the organization — and to architects themselves. As organizations scale, both the system and the architectural role evolve through stages:

- **Infancy (startups / early products):** Strong central guidance establishes patterns, standards, and prevents early chaos.
- **Adolescence (scaling teams and domains):** Teams gain independence within clear guardrails. Architects shift from decision-makers to coaches — challenging assumptions and enabling safe autonomy.
- **Adulthood (mature enterprise):** Autonomy becomes essential. Architecture focuses on designing the enabling system — shared principles, platforms, and feedback loops — rather than directing individual decisions.

Figure below highlights the different stages of an evolving architecture autonomy aligning with an evolving organization.

Life Stages of a System: Evolution of Architecture Autonomy

From Centralized Control to Decentralized Autonomy with Guardrails

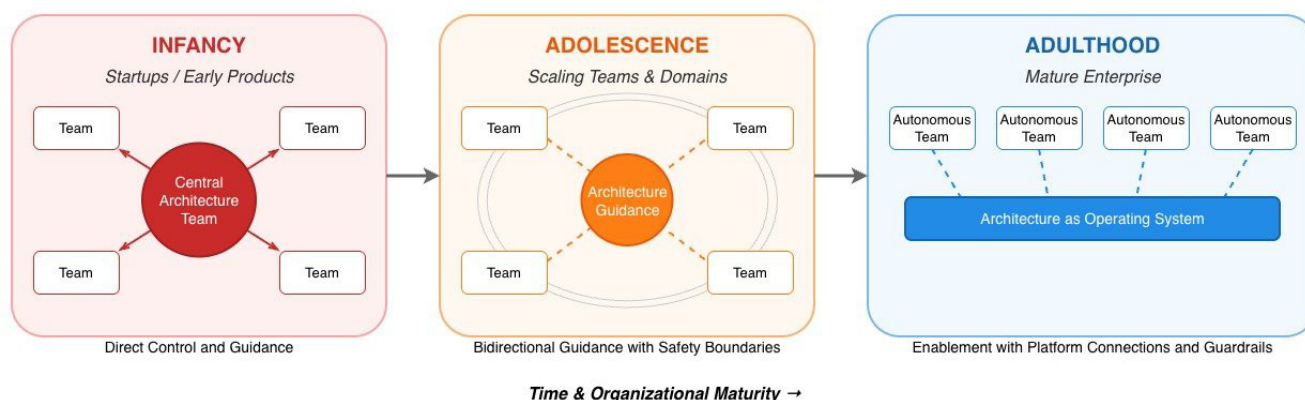


Figure 1: The evolution of architectural governance from centralized “Infancy” to decentralized “Adulthood”

As organizations mature, architects need to evolve the methodologies as well. Below depiction highlights such evolution in a growth framework and evolving architecture autonomy as the organization matures.

The real risk is not decentralization, but remaining stuck in a control mindset after the system has outgrown it.

This article proposes a *structured framework* for progressing toward responsible decentralization. It is built around several core elements:

- Clearly defined guardrails
- Explicit decision scopes and boundaries
- Well-articulated architectural principles and guidelines
- A disciplined practice of preserving architectural decision history
- Architecture Alignment Forum
- Platform department as an enabler for scale and velocity
- Outcome-based KPIs and fitness functions
- Cultural adoption of the architect’s role

Architecture Governance Evolution: From Control to Enablement

Decision Boundaries, Principles, and Platform Enablement Across Organizational Maturity

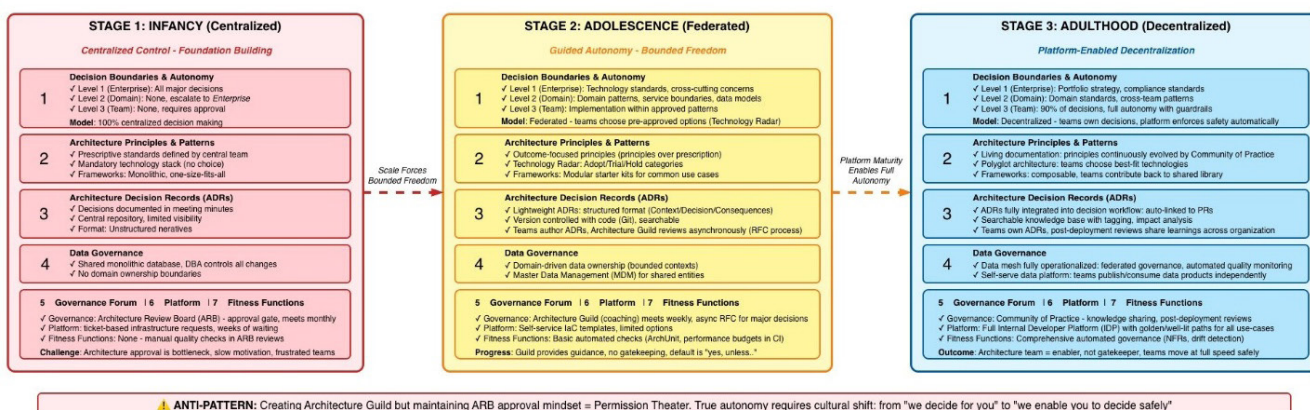


Figure 2: Architecture Governance Evolution: From Control to Enablement

In this article we'll cover in detail each of these core components as well as the role of AI an enabling superpower.

Guardrails in Practice: Enabling Autonomy Responsibly

In centralized models, architecture often acts as a gatekeeper. Clear architectural standards prevent analysis paralysis and eliminate costly distractions especially for startups which are often resource-constrained and under time-critical deliveries. With limited room for experimentation where failure carries existential risk, centralized guidance provides the directional clarity needed to align teams toward a unified goal. This disciplined focus significantly increases the probability of successful outcomes during the critical early-stage period.

When that gate becomes slow, distant, or overly controlling, teams respond predictably: under delivery pressure, they bypass processes, duplicate solutions to reduce dependency, introduce undocumented patterns, and accumulate hidden technical debt.

This is why the architect's role must shift from enforcement to friction removal. In a healthy decentralized model, teams are allowed to "fall in the backyard" – to experiment within defined safety boundaries. The aim is not to eliminate failure, but to prevent catastrophic failure. Trust is built not through control, but through clear guardrails, transparency, and reliable feedback loops.

"The architect has to focus on communication, removing frictions and a coherent understanding of the system's purpose and design across all participants—and not necessarily act as a gatekeeper or law enforcer."

- Pascal Euhus¹

Decentralising architecture becomes successful when guardrails are explicit, lightweight, and scalable. Let's examine some aspects of a decentralized approach.

Decision boundaries: autonomy at each architectural level

One practical way to decentralise responsibly is to align architectural authority with levels of abstraction: In the context of C4 model it may look something as depicted below:

Scope	IC Role
Context (System)	Enterprise / Portfolio Architect
Containers	Senior Solution/ Software Architect
Components	Solution Engineer / Junior Architect
Code	Tech Lead and Development Teams

This reflects a simple reality: local teams often understand low-level intricacies best, while portfolio architects better understand cross-domain integration, strategy, and enterprise constraints.

Architecture principles and guidelines: autonomy without anarchy

Architecture principles and guidelines enable distributed decision-making by acting as a shared social contract within the engineering organization. They clarify values, trade-offs, and boundaries – allowing teams to operate autonomously without constant central approval. Without shared principles, distributed decision-making risks gradual divergence. When standards are too rigid, teams default back to centralized approval. Well-defined principles, paired with adaptable guidelines, help maintain coherence while allowing teams to operate independently.

Consider the principle:

Data is a strategic organizational asset and must be managed, protected, and governed accordingly.

From this follow practical guidelines:

- Every dataset must have a defined owner
- Owners are accountable for quality, lifecycle, and access control
- Critical data elements must define measurable quality metrics
- Schema changes must follow versioning rules
- etc.

Guidelines translate intent into behavior. They answer a practical question: if data is strategic, what does responsible action look like? By making expectations explicit, accountability shifts to teams — and architectural alignment no longer depends on centralized decision-making. Without the principles and guidelines in place, different teams may have their own definition of what good data looks like, resulting in inconsistency across the organization.

Architecture Decision Records (ADRs): preserve intent

Architectural decisions often present multiple viable paths, each with trade-offs. In choosing one, we rely not only on analysis but also on accumulated experience. An Architecture Decision Record (ADR) provides a structured way to document that choice.

ADRs are *lightweight documents* that capture decisions alongside their context, rationale, alternatives, and consequences. Their purpose is to *preserve intent* — to make explicit why a particular path was chosen at a specific moment in time.

As systems evolve, decisions are revisited, refined, or superseded. ADRs create a living record of those choices, supporting both active development and long-term maintenance. They help answer the recurring question: “*Why was it built this way?*” Without them, context fades and architectural reasoning becomes tribal knowledge.

Maintained at the team or domain level, ADRs provide traceability that enables safe experimentation and informed autonomy. Complementary practices such as *Requests for Comments (RFCs)* extend this model by inviting decentralized feedback before decisions are finalized, strengthening alignment without centralizing authority.

Decentralized decisions must still be traceable

A core value of ADRs is that they prevent future teams from inheriting code *without context*. Even in a decentralized organization, decisions should never become invisible. ADRs ensure that autonomy does not come at the cost of traceability—and that architecture remains both distributed and coherent over time.

The Collaborative ADR Review

ADRs are most valuable not as approval checkpoints, but as forums for thoughtful discussion. When approached well, the review process resembles a family meeting more than a compliance exercise — guided by mentorship rather than authority. Its purpose is not to grant permission, but to strengthen architectural judgment.

Review questions might include: What trade-offs were considered? How would this decision behave under scale or failure? What risks does it introduce for adjacent domains? How reversible is it?

Such dialogue builds decision-making muscle. Engineers learn to internalize architectural reasoning rather than defer to central authority. Over time, dependence on senior architects decreases as teams gain confidence in evaluating complexity themselves. ADR reviews, then, are not governance rituals but learning mechanisms — cultivating distributed judgment rather than centralized control.

Architecture Governance Forum: escalation and alignment, not permission

An Architecture Governance Forum brings together IT, data, and business stakeholders to coordinate strategy and execution. It acts as a bridge between strategic intent and technical delivery. While it may appear centralized, a well-designed forum enables decentralization. Rather than controlling decisions, it provides alignment, guardrails, and a clear escalation path when cross-domain trade-offs arise.

When operating effectively, the forum helps teams move faster and with greater confidence. It creates space for constructive challenge, surfaces risks early, and ensures decisions align with enterprise principles and long-term direction. It also helps build cross-stakeholder buy-in for significant architectural choices.

Importantly, the forum should not function as an approval gate or an *ivory tower*. Its role is to clarify constraints, context, and the organization's *North Star* — empowering teams to innovate while remaining aligned with shared standards and strategic direction.

Build the Platform - Scale the Autonomy

Decentralized autonomy does not scale by intention alone. It requires deliberate investment in shared capabilities. Without a strong platform, autonomy leads to duplication, fragmentation, and hidden risk. A well-designed platform turns constraints into reusable building blocks and guardrails into embedded defaults.

The operating model is straightforward:

- **Centralize cross-cutting capabilities** — security, data privacy, shared infrastructure, developer tooling, and governance automation.
- **Decentralize domain decisions** — service design, integrations within defined constraints, and localized delivery.

Platform teams create the technological foundation — the “paved road” of reusable infrastructure, CI/CD pipelines, observability, and identity controls. When teams are under delivery pressure, they will naturally choose the path that feels fastest and least obstructive. If the platform is designed well, that path is also the safest one.

This logic extends beyond engineering. Functions such as Data Governance can operate as shared platforms, offering tooling for data quality, metadata, and stewardship, while domain teams retain ownership of their data within enterprise standards.

When designed well, platforms embed alignment into systems rather than meetings. They make decentralization sustainable by reducing friction while preserving coherence.

Outcome-based KPIs and fitness functions

In a decentralized model, the shift is subtle but important: instead of controlling how teams work, the focus moves to validating what they produce. As decision-making moves closer to teams, the system still needs something that preserves coherence. Clear boundaries serve that role, protecting the integrity of the whole while autonomy operates locally. Those boundaries — the guardrails — must be clearly articulated, objectively verifiable, and embedded into the platform so they apply consistently across teams.

This is where **fitness functions** become essential. A fitness function is an automated check that continuously validates a system property, ensuring architectural characteristics remain intact as the system evolves. Rather than reviewing architecture only at design time, fitness functions evaluate it during build, deployment, and runtime.

Examples include:

- Failing deployments when security vulnerabilities exceed defined thresholds

- Enforcing measurable data quality targets for critical datasets
- Monitoring adherence to SLO and availability commitments
- Validating compliance with data classification or encryption policies

While KPIs define *intent*, fitness functions verify that intent continuously *in practice*. When guardrails are encoded into pipelines, platform tooling, and observability systems, alignment shifts from periodic review to ongoing validation. Decentralization remains sustainable only when constraints are structural, observable, and continuously enforced by the system itself.

To summarize in the context of growth framework, some of the core components discussed above may not/may exist in some shape and form during early stages. But as organization matures these processes may be further evolved to empower teams and enable architecture autonomy .

Decentralization can have profound impact on not just ways of working for teams but also attaining greater operational efficiencies.

The table below summarizes the impact of successful operationalization of decentralization, as the organization navigates through the growth framework from Infancy stage to Adulthood.

Ceremonies of Connection: Moving from Commander to Coach

Decentralized models fail when teams drift into silos. They succeed when shared rituals preserve context and trust. As discussed earlier, ADR reviews and Architecture Forums function less as approval gates and more as collaboration rituals. But these forums are only part of a broader cultural shift — one that reshapes the architect's role and the organization itself.

The Socratic Architect

In a decentralized organization, the architect's role shifts from approver to coach — guiding teams through trade-offs, constraints, and long-term consequences. Rather than sitting atop a decision hierarchy, the architect works alongside teams, helping sharpen reasoning. The aim is not to centralize decisions, but to elevate judgment.

Instead of prescribing solutions, the Socratic architect asks questions: What trade-offs are being optimized? What assumptions underlie this

Core Component Evolution

Guardrail	Infancy	Adolescence	Adulthood
1. Architecture Standards	Mandatory stack (limited to approved tools / technologies)	Technology Radar	Polyglot + Fitness Functions
2. Data Governance	Shared monolithic DB	Domain ownership + MDM	Data Mesh + Self-serve Platform
3. Review Process (Governance Forum)	ARB - Approval Gate (may not exist)	Architecture Guild - Coaching Forum	Community of Practice - Knowledge Sharing
4. Platform Services	Ticket-based provisioning (may not exist)	Self-service IaC templates	Internal Developer Platform (IDP)
5. Fitness Functions	Manual quality reviews	Basic automated checks (ArchUnit, perf budgets)	Comprehensive automated governance (NFRs, coupling, drift)

design? How does it behave under failure or scale? How reversible is it? Who bears the operational burden? Such questions may feel uncomfortable — not because they limit autonomy, but because they expose hidden assumptions. They build decision-making muscle, helping teams internalize architectural discipline rather than rely on central authority. Over time, dependence decreases as judgment becomes distributed. Control gives way to inquiry; authority to capability-building.

The Village Effect

Decentralization does not remove the need for connection — it heightens it. As authority disperses, shared memory becomes essential to coherence. Communities of practice, cross-domain forums, and recurring exchanges allow patterns and lessons to circulate across teams. While ADRs document decisions, communities preserve the experience behind them.

As knowledge spreads, reinvention declines and autonomy gains context. Teams operate independently, yet remain aware of the broader system. At scale, this connective rhythm becomes critical. Cadence-based planning and stakeholder syncs provide visibility into dependencies early, reducing surprise and unnecessary escalation.

In a commander model, coordination depends on authority. In a village model, it grows from shared understanding. Autonomy is not isolation, but independence within a consciously connected system.

AI Enabling Decentralized Architecture

AI is often associated with code generation, yet its deeper value lies in strengthening architectural autonomy. When used within clear guardrails and responsible governance, AI helps teams make informed local decisions while maintaining organizational coherence. Rather than centralizing authority, it amplifies visibility and judgment.

Design and Decision Support

AI can act as a design-review copilot, surfacing gaps across security, resilience, privacy, and compliance based on established principles and patterns. It can suggest alternatives, structure trade-offs, and assess alignment with internal standards.

It also accelerates common architectural comparisons — distilling vendor documentation, summarizing internal runbooks, and generating structured inputs or draft ADRs for refinement. This reduces research time while preserving human oversight.

Organizational Memory and Dependency Awareness

By analyzing ADRs, repositories, roadmaps, and release notes, AI can surface decisions that affect adjacent teams and highlight cross-domain dependencies. What was once scattered across artifacts becomes visible in context, reducing surprise and improving coordination.

Guardrails and Drift Detection

AI can continuously scan code, infrastructure, and runtime signals to detect deviations from agreed principles — insecure patterns, missing data lineage, improper handling of regulated data, or unapproved integrations. By correlating platform standards, ADRs, and configuration changes, it surfaces misalignments earlier than periodic reviews would allow.

The objective is not tighter control, but earlier awareness. Teams retain autonomy, while deviations become visible before they turn into systemic risk.

Enabling Distributed Judgment

Taken together, these capabilities change how autonomy scales. AI does not replace architects or centralize decisions; it strengthens the environment in which decisions are made. By linking principles, platforms, and real-time feedback, it helps distribute architectural judgment more broadly across the

organization. Autonomy remains local — but it operates within a system that is increasingly informed, transparent, and resilient.

The Approachable Architect

Decentralizing architecture does not mean removing architecture. It means evolving it into a scalable operating model — where guardrails, platforms, principles, and feedback loops replace centralized bottlenecks. The most effective architects are those who gradually make themselves unnecessary for day-to-day decisions — not because architecture disappears, but because it becomes embedded in the way teams work. Teams operate with clear principles, shared patterns, paved roads, and governance they trust.

Sustainable decentralization, however, depends on more than structural design. It is ultimately an organizational transformation, not merely a technical one. It requires sustained leadership commitment and alignment across levels of the organization. Executives must actively

champion the shift from approval-based to trust-based governance, investing not only in technical platforms but also in capability building, communities of practice, and cultural evolution.

This transition demands realism. Autonomy comes with intelligent failures. As teams assume operational ownership, incident rates may temporarily rise before stabilizing. Maturity varies across teams — some are in infancy, others in adolescence, a few in adulthood — and governance must adapt accordingly. Granting premature autonomy to teams without operational discipline invites chaos; imposing excessive control on mature teams drives shadow systems and undocumented workarounds. Technology provides the tools, but sustained organizational alignment ultimately determines whether those tools deliver meaningful and lasting results.

Sources:

¹[Software Architects and Autonomous Teams](#)



Decentralized Architecture in Practice: Decision Rights, Organizational Patterns, and How AI Amplifies What Is Already There

Steven O’Kennedy, Director of Architecture
Andreas Hoersken, Software Engineer
Wejdan Bagais, Senior Machine Learning Manager
Pushpdeep Mishra, Senior Engineering Manager

Introduction

Early in a project, a small group can design, decide, and deliver quickly. You gather people, sketch boundaries and integrations, make key decisions - and the system gets built. As the product and the organization grow, however, that concentrated attention becomes harder to sustain. Priorities pull people away, interactions multiply, and the knowledge needed to manage the landscape fragments across teams. What once fit in a few heads becomes brittle under scale.

This article outlines why decentralizing architectural decisions is often necessary for systems to remain adaptable and reliable as organizations grow, how to implement it successfully, and why it is hard for companies to do so. A **decentralized architecture** is a model in which **decision rights and responsibility for design are distributed** to teams working within **shared, organization-wide guardrails**, so that changes can be made close to the work while **preserving system-wide coherence**.

We'll explain why centralization persists, identify the organizational, technical, and human barriers you're likely to encounter, and describe practical patterns for evolving toward a decentralized approach without sacrificing coherence or quality.

The guidance is aimed mainly at architects working in larger companies facing scale and coordination challenges, but many of the patterns apply to teams of any size. Throughout the piece, we'll focus on what "good" looks like in decentralized architecture practice and how you can move toward it deliberately.

I. Why Decentralized Architecture Is Hard (Even Today)

The problem that is described above is recognizable as a pattern to many who have been involved in the software industry for any length of time. It has been visible ever since we started building large interconnected software systems, from the 1960s with early mainframe designs to the present day. It would seem that there should be an easy solution to this problem. A typical approach that has been used successfully for decades to reduce the complexity of software engineering problems. Divide and Conquer. Instead of relying on a central group to manage decision-making, analysis, trade-offs, and solutions related to the system architecture, decompose the large problem into smaller problem spaces and distribute the work to solve simpler, smaller problems. But it's never as simple as that.

If it were, why do we still see so many instances of centralized architecture teams, centralized decision-making, and centralized architecture governance in modern businesses of all sizes today? Well, firstly, because decentralizing has its own costs, e.g., duplicated effort or risk of inconsistencies, that often means that a centralized approach is the rational one until a company has reached a tipping point where the problems outweigh that cost. Secondly, it's not easy. Our view is that, for a large company, three main drivers or blockers make

this a hard change for companies to implement successfully:

Driver/Blocker	Can Lead To
Organization Structure and Norms	Decision latency and resource bottlenecks
System Boundaries and Scopes of Control	Coordination overheads and delays
Human Behavioral Patterns	Resistance to Change

Organization Structure and Norms

The size of the organization and its growth stage create different obstacles to consider. Startups and small companies don't typically suffer from centralized architecture because their priorities are survival, growth, and customer needs (often in that order!), and their limited staff, by nature, usually makes them decentralized with a flat hierarchy.

Large organizations rarely have the luxury of staying flat. As the enterprise grows, hierarchy deepens, and complexity rises, so a common response is to centralize responsibilities within dedicated architecture teams or similar central groups.

At first, this appears to work: it concentrates skills, enables clear communication among architects across large application landscapes, and keeps decision-making within a small, manageable group. But over time, it becomes unwieldy and slows things down. It creates distance between decision-makers and the teams implementing the solutions (and, most importantly, between them and the users of those systems), producing siloed/ivory-tower behaviors: capacity bottlenecks, lack of pragmatism, and disconnection from reality. Worse, it can exacerbate the problem it was designed to solve, as it can remain difficult to scale when a small group controls decision-making.

System Boundaries and Scopes of Control

The way that team and architecture responsibilities are divided between systems or components - and how those boundaries align with the parts of the

organization that own, contribute to, and support them - directly affects how easy or difficult it is to decentralize the architecture effectively.

The more overlap there is across system and component boundaries, the more architects and teams need to coordinate before any action can be taken. If a team's scope of responsibility/authority overlaps or depends on other teams, it greatly complicates that team's ability to make architectural decisions. The more overlaps there are, the worse it gets.

Conway's Law states that an organization's output will mirror the shape of its communication structures. Decentralizing the architecture requires not only architectural change, but also similar organizational changes at the team, budget, and oversight levels within the organization. This is a cost that can be difficult to prioritize in the face of pressing daily needs for daily support and key additional functionality.

Human Behavioral Patterns

The idea of decentralizing core decision-making processes and architecture can be very frightening to those who currently hold decision-making authority. Command-and-control structures still persist today, even in companies that attempt to align with Agile practices. The thought of laying down this power can feel very intimidating, even for the best-intentioned, if they believe there are skill gaps or the risk of office politics. Sometimes this is indeed justified, and those with control may be best placed to make decisions, but it does limit the growth potential.

Historically, human behaviors have often shown that central approaches are easier to get buy-in for than decentralized ones. Look at any example of an adopted vision from a charismatic leader. Decentralizing is harder because it requires a shared vision, mutual trust, and a clear split of responsibility. Change toward this requires

conscious effort, even in the best-case scenario when everyone agrees on the same way forward.

The remainder of this article explains what good looks like in practice (observable signals), the organizational patterns that make it possible, and how AI can help to amplify impact and ease the transition to a decentralized approach.

II. What "Good" Looks Like in Modern Decentralized Architecture Work

Modern software systems operate in environments of constant change. Teams are increasingly autonomous, and decisions are made faster than any single individual can oversee. In this context, architecture is no longer a phase at the beginning of a project or a static set of diagrams.

Instead, architecture shows up continuously - in how and why decisions are made, how change is absorbed, and how understanding is preserved as complexity grows. This view aligns with the idea of *continuous architecture*, where architectural work evolves alongside the system rather than being frontloaded at the start of a project (Bittner and Pureur). Crucially, decentralization does **not** mean decentralizing everything. Shared rules, patterns, technologies, and processes must remain centralized as guardrails. Without this harmonization, decentralized decision-making becomes blocked by inconsistency and fragmentation. What should be decentralized are **individual decisions**, not the constraints within which teams operate.

(Ref: Continuous Architecture with Kurt Bittner and Pierre Pureur: <https://www.infoq.com/podcasts/continuous-architecture/>)

Defining the End State: What Success Looks Like

Early technical choices or shortterm milestones do not define success, but by how the system and teams behave over time. Getting to the destination won't be a single-step journey. As with any move toward a north star, some interim targets need to be

set, including an assessment period during which progress may slow, and mistakes are expected. Here, changes can be assessed and refined, inevitable mistakes corrected, and the path forward identified.

Progress Is Intentional, Not Reactive. Architecture gives change a direction. Teams move forward without repeatedly reopening the same decisions or undoing earlier work. Change moves smoothly through the system; it is absorbed deliberately rather than creating friction or confusion. Over time, this results in more predictable delivery because the system no longer resists change.

Ownership Enables Independent Movement. Responsibilities are explicit. Teams know where their accountability starts and ends, allowing decisions to be made close to the work while escalating appropriately when shared guardrails or patterns need to evolve. This enables autonomy without sacrificing alignment.

Intent Outlasts Individual Decisions. Decision rationale, tradeoffs, and constraints remain visible over time. This shared understanding reduces friction as teams grow or change, allows new contributors to orient quickly, and ensures decisions age well even as details evolve.

When all of this is achieved, the resulting architecture is not only decentralized, intentional, and communicable, but also composable.

Observable Qualities of a Composable Architecture

Architecture is never “done.” The goal is to enable adaptation under uncertainty while maintaining coherence. If the end state describes what success feels like over time, the next question is more practical: how do you recognize progress while the work is still unfolding? Good architecture rarely announces itself upfront, and composable, decentralized architecture even less so. Instead, it becomes visible through a set of everyday signals.

These are not checklists or maturity models. They are observable qualities reflected in how teams work, how decisions are recorded, and how change behaves.

Boundaries Are Clear and Respected. Teams know what they own and how others interact with it. Even under pressure, accountability remains with the owning team - this is ours to fix, explain, and learn from - enabling independent evolution without disrupting the wider system.

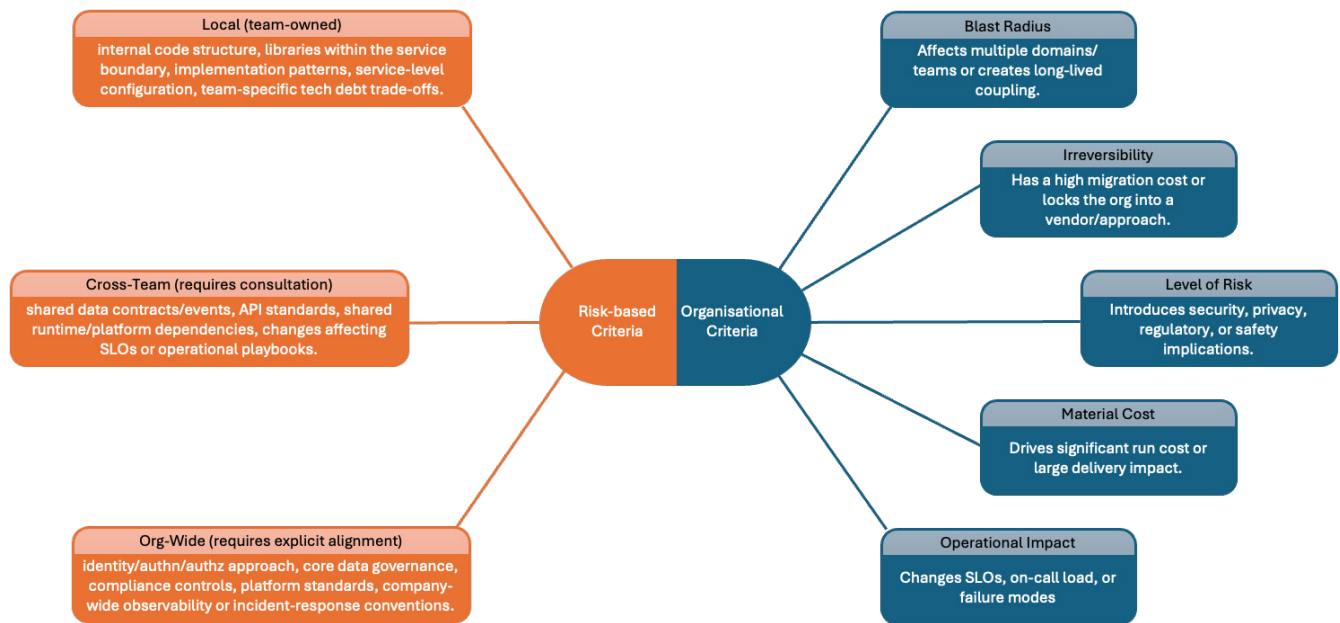
Visible Decisions and Shared Understanding. Important decisions are easy to find and easy to understand. Teams do not rely on tribal knowledge to explain why the system looks the way it does. Visible intent prevents divergence and allows teams—new and existing—to continue work without relying on unwritten context or rediscovering past decisions.

Change Has a Limited and Understandable Impact. Another strong signal is how change behaves in practice. In healthy systems, changes affect a limited and expected area. Teams can evolve their part of the system without routinely disrupting others.

III. Organizational Patterns That Make This Work

While the best path to decentralized decision-making depends on your organization’s starting point, there are recurring patterns that reliably produce the outcomes described in Section II (clear ownership and boundaries, visible decisions, and change that stays contained). They fall into three categories: establishing clarity, enabling good decentralized decisions, and sustaining the process over time.

In this section, decentralization means distributing decision rights - and the responsibility to make timely, reasonable, and transparent decisions - within shared constraints. We assume development happens across one or more teams. Decentralized decision-making means that each team (or even



an individual developer) can make and drive architectural decisions as needed to get their work done.

Establish Clear Goals and Responsibilities

Before any decision can be decentralized, the above-mentioned fundamentals must be in place. Teams and leadership need a shared understanding of the rationale behind decentralization, what good architecture looks like, where it shall take place, and who is responsible.

The “who” can be defined at different levels of granularity. In most implementations of decentralized architecture, the people who need an architectural or technical change drive the decision.

To help teams with what to decide and in which context, it helps to define criteria - for example, based on organizational structure or the criticality of the decision, as described in the diagram above.

A simple rule works well: the wider the blast radius and the harder it is to reverse the change, the more structured the guidance and alignment should be.

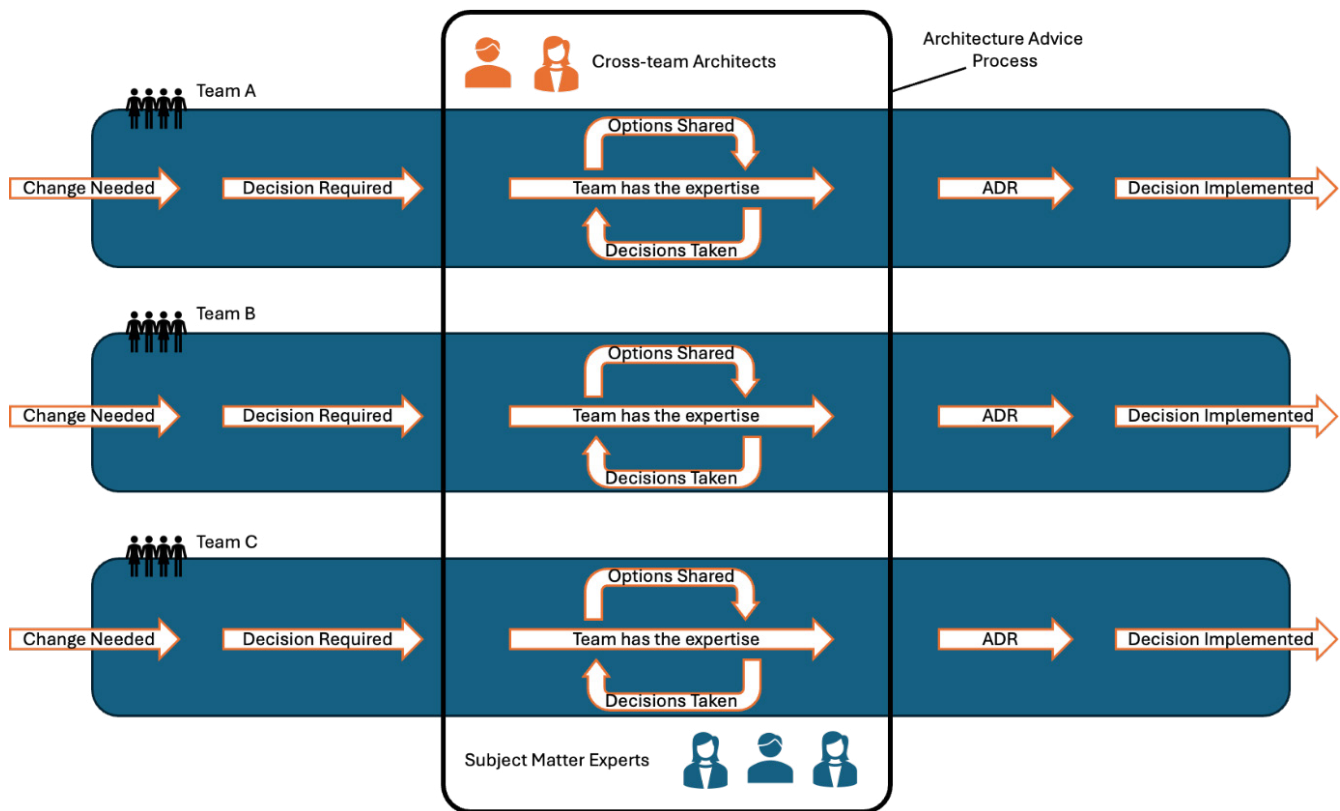
Enable Good Decisions

Decentralizing authority without enabling good decision-making is a recipe for chaos. Organizations need to invest deliberately in the structures, processes, and knowledge that enable teams to make sound architectural choices.

Define and document the process. A well-known pattern is the Architectural Advice Process (Harmel-Law 2021): any team can make any architectural decision, as long as they seek advice from all relevant parties and experts. The decision, its context, rationale, and any feedback received must be recorded and shared publicly - typically as an Architecture Decision Record (ADR) (Gonchar 2023).

Establish architectural principles. Principles rooted in business needs serve as guardrails. They clarify not only when and why decisions are made, but, more importantly, how decisions should be approached.

Provide access to expertise. Not every team has the same depth of architectural knowledge - and that is fine. Organizations can bridge this gap by establishing experts for specialized topics (e.g., UI/



UX or architecture), platform teams for enabling areas (e.g., infrastructure or DevOps), or setting up cross-team forums (e.g., communities of practice, architecture advisory forums). Crucially, these experts should not make decisions themselves (or alone); they should support and enable the people who are responsible.

Sustain and Support Decision-Making

Enabling decentralized decisions is not a one-time effort. It requires ongoing organizational commitment.

Create a safe environment for experimentation and growth. Especially at the beginning of a transformation to decentralized decision-making, uncertainties and even mistakes can occur. A gradual transfer of decision-making authority - based on the criticality of the decision - or stronger support can limit the impact of wrong decisions.

The goal here is not to eliminate failure, but to eliminate catastrophic and hidden failure.

Live with the decisions made. Management and architects must accept decisions made by teams. Frequently reversing or overruling decisions is deeply damaging - it strips teams of responsibility and erodes trust in the process. It is far better to invest in providing the necessary expertise and knowledge before a decision is made than to correct it after the fact.

Encourage ownership. Make ownership explicit, and acknowledge accountability for it. Recognize and reward teams for owning outcomes, reflecting on impact, and improving based on what they learn.

Allocate time and resources. Making good architectural decisions - and building the knowledge required for them - takes time. This must be acknowledged and planned for, especially

during the initial rollout when everyone is still learning the process. The purpose and value of architectural decision-making need to be clear to all stakeholders (including product owners and business stakeholders) so that the necessary time is respected rather than squeezed out. Support from experienced knowledge holders should be planned explicitly.

Antipatterns to Watch For

Even with the best intentions, organizations can fall into traps that undermine decentralized decision-making:

Decisions are not being made transparently.

Ambiguity and opacity are just as harmful as over-centralization. If decisions are not documented and shared, other teams cannot build on them. Standards for where and how to document decisions should be in place. Filing ADRs early for review (or a request-for-comment phase) in an accessible wiki or repository has proven successful here.

Making it hard to make decisions. Complicated, overloaded, or unclear processes and procedures make it difficult for teams to decide efficiently and increase the likelihood that processes are bypassed.

Decisions are not being made, or teams are becoming communication silos. When teams ignore decisions - either their own or those made by other teams - because they don't seem relevant right now, architectural coherence suffers. Actively fostering cross-team accountability and clearly communicating the objectives and added value of architectural decision-making reinforces responsibility. Ultimately, however, it also depends on a sense of responsibility and a certain level of seniority among teams and individuals. That can be developed over time.

Injecting external requirements without ownership. Global or external requirements (e.g., ISO certifications or audits) are often decided in

a closed loop by those with overall responsibility. Treat them like any other architectural decision: same transparency, and the same expectation to write ADRs and share them with teams - otherwise skepticism and disengagement grow.

Blocking or long-running decisions. Decentralized decision-making is undermined if decisions are blocked or take too long. This risk should be addressed explicitly in the process description. In practice, two proven patterns help: using consent-based (rather than consensus-based) decisions, and explicitly assigning responsibility to a decision driver. Guidelines for decision timeframes can be graded in the same way as the heuristics for what needs to be decided (organizational vs. risk-based approaches). Ultimately, the decision driver should ideally be allowed to determine the timeframe for the decision.

As AI increases the rate of change and the volume of decisions, these patterns become even more critical.

IV. When AI Amplifies Decentralized Architecture - for Better or Worse

The points described earlier in this article apply to any system, but with AI augmentation dramatically accelerating the ways and places where decisions are made, decentralization becomes unavoidable. Modern AI-assisted approaches to work do not create new architectural problems, but they do amplify whatever the existing architecture already enables - for better or worse.

When boundaries are clear, decisions are visible, and change is contained, AI accelerates progress. Teams explore more options and reduce friction. When those qualities are absent, the same tools accelerate confusion, inconsistency, and drift. In this sense, AI acts less like a new capability and more like a stress test for your domain boundaries, decision-making processes, and architectural guardrails.

AI Pushes Decisions Closer to the Work Regardless of Readiness

AI tools increasingly support local decision-making by suggesting approaches, generating alternatives, and summarizing tradeoffs. This shifts architectural decisions even closer to where the work happens. If you are not prepared for decentralization, then AI acceleration will expose that. AI-assisted choices may look reasonable in isolation while increasing coupling or reducing cohesion over time. The issue is not the tool, but the lack of architectural constraints and structures.

Preparing for decentralized decision-making with clear decision criteria (local vs. cross-team vs. org-wide), a documented rationale, and visible guidance such as architectural principles and ADRs, is a must. AI can then use the accumulated record of “how we decide” and “what we have previously decided” as a working context, guiding the AI toward better local decisions while making those decisions significantly faster, safer, and easier.

Architectural Drift and Loss of Understanding Accelerate Together

One of AI’s biggest risks is not sudden failure, but the gradual erosion of the architecture. Small, locally reasonable but globally problematic changes can accumulate faster than before. More concerning, systems may continue to function while teams lose the ability to explain how they are aligned, what their boundaries are, or even why they behave as they do. When architectural intent is implicit, understanding erodes, incidents become harder to reason about, and confidence in change declines.

In AI-augmented systems, preserving intent and decision rationale is not optional. It is what allows teams to reason about the system, rather than merely operate it.

Emergent Values after AI: Clarity is More Valuable Than Speed

As AI lowers the cost of producing code and documentation, the cost of making the right

decisions remains. Architecture is still, as [Ralph Johnson said](#), “the decisions that are hard to change.” The value of architecture in this decentralized, AI-augmented world, therefore, shifts away from just prescribing solutions toward **making intent explicit**. To put it another way, “When speed becomes cheap, judgment carries a premium,” as was succinctly put by [Ann Handley](#) in her [recent article](#).

Clear boundaries, documented tradeoffs, and understood constraints provide the context both humans and AI need to stay aligned. With that, AI can help teams answer questions quickly, generate high-level overviews, and brainstorm new decisions faster. It gives engineers more time to communicate and collaborate with other parts of the business to reinforce and correct that context. It also helps architects ensure that the context remains in place. But without it in place at the start, then AI output may still appear correct while quietly destroying a system’s longterm shape, core boundaries and ultimately its maintainability.

Conclusion

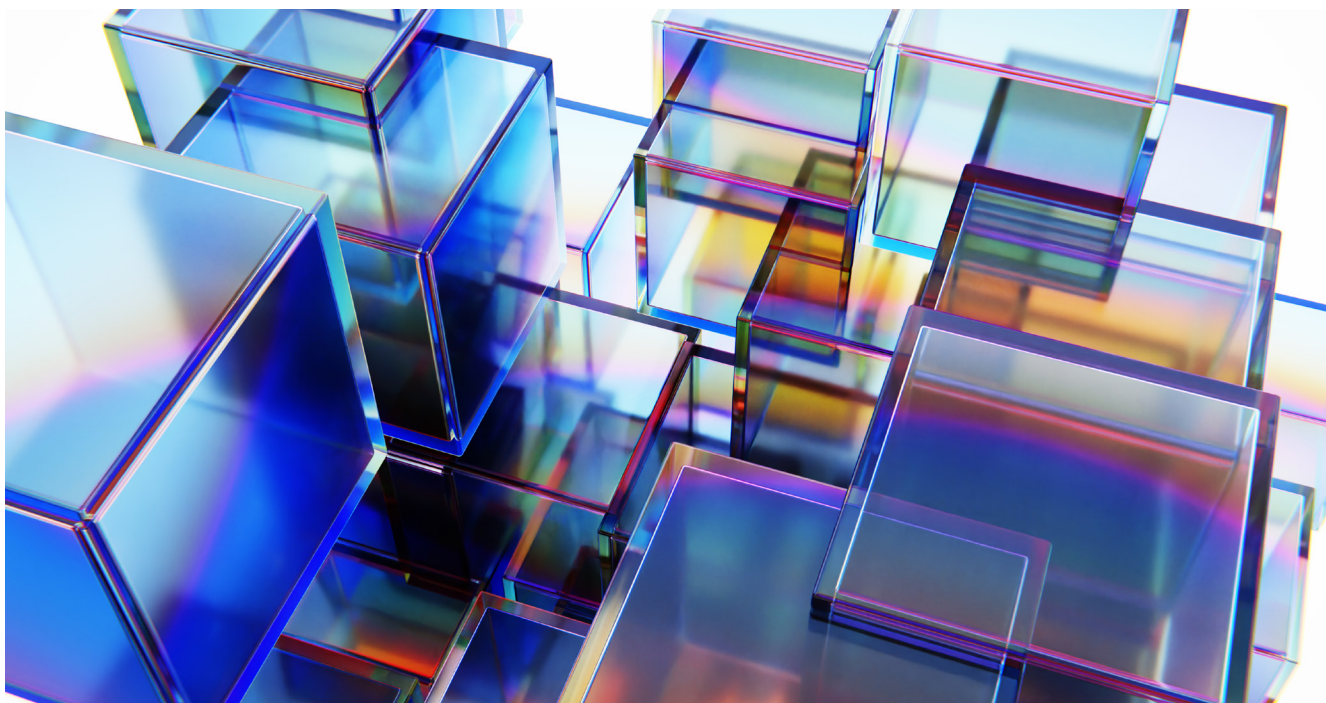
We’ve shown how decentralizing architecture changes what good architecture work looks like. Designing systems and choosing technologies still matter, but they are no longer the primary measure of success. What matters most is making architecture composable and making context explicit: shaping how decisions are made over time, enabling those decisions to be taken as close to the work as possible, and keeping system boundaries clear so change can be absorbed with a limited blast radius.

The hardest work is organizational, not technical. Tools, platforms and AI can accelerate outcomes - for better or worse - but they don’t solve the human and governance problems that determine whether a decentralized model succeeds. Leaders must shape structures, allocate resources, embed decision-making practices, and align incentives so teams

can make highquality, autonomous decisions and preserve the intent behind them.

AI amplifies both strengths and weaknesses. When boundaries, visibility and feedback loops are in place, AI delivers disproportionate gains; where they are missing, it accelerates drift. That makes the work of documenting rationale, enforcing guardrails, and investing in fast feedback and observability more urgent, not optional.

In short: aim to decentralize authority but centralize the rules of engagement. Do the organizational work first - clarify ownership, capture intent, and build lightweight governance - and let teams, supported by AI, move fast without fragmenting the system. That is how architecture remains adaptable, coherent, and resilient in the age of AI.



Architecting Autonomy: How Decentralizing Architectural Decisions Enables Organizational Scale

Alex Hakmeh, Senior Technology & Delivery Leader
Ajeesh George, Solution Architect
Ruth Montgomery, Senior Solution Architect

The Traffic Officer and the Roundabout

Picture a busy junction at rush hour. Traffic converges from four directions. A single officer stands in the center, manually deciding who moves next. At first, the system feels orderly. It is controlled. It is predictable. But as volume increases, queues form. Drivers wait longer. Frustration rises. The officer becomes overwhelmed. The entire system's capacity is limited by a single individual's ability to direct the flow.

Now imagine replacing that junction with a roundabout. There is no officer. Instead, there are shared rules, clear lines of sight, and distributed decision-making. Each driver decides when to enter based on context and agreed conventions. The system flows not because it is centrally directed, but because it is structurally designed for concurrency.

The metaphor is not about roads. It is about scale.

As organizations grow, they encounter the same dynamic. Architectural decisions increase in

frequency and complexity. If every significant decision must pass through a central authority, the organization's growth is constrained by the cognitive capacity of a small group.

Scaling an organization is fundamentally a concurrency problem. Organizations don't scale by adding people; they scale by increasing decision throughput. Decentralizing architectural decision-making is what unlocks that throughput, especially as domains multiply, the number of architectural interfaces grows combinatorially, and coordination costs rise faster than headcount.

Decentralization as a Scaling Strategy

Centralized architecture creates coherence at a small scale. It reduces duplication, enforces standards, and accelerates early progress. But as domains multiply and teams expand, the volume of architectural decisions increases dramatically. New services, integrations, platforms, regulatory requirements, and customer features all introduce decision load. The number of architectural decisions grows non-linearly with organizational complexity and when those decisions remain centralized, a queue forms, latency increases, feedback loops lengthen, delivery slows, and teams disengage. The architecture function becomes the bottleneck.

This is not a matter of discipline or competence. It is a structural consequence of scale, as no central body can absorb that increase indefinitely. Dependency does not scale.

In systems terms, centralized decision-making introduces tight coupling and singlethreaded throughput into what is otherwise a distributed system. As concurrency increases, the bottleneck becomes visible. Decentralizing architectural decisions distributes authority closer to information. It increases parallelism. It reduces decision latency. It allows domains to evolve simultaneously rather than sequentially.

Decentralization is not primarily about empowerment. It is about throughput. Architecture does not fail at scale because standards are weak. It fails because decision throughput cannot keep pace with organizational growth.

A Real-World Scaling Inflection Point

In a regulated payments platform expanding into new markets and product lines, architectural decision-making remained centralized long after the organization had outgrown its early-stage operating model. Every significant service boundary change, integration decision, and data model adjustment required review by a central architecture forum, a TDA (Technical Design Authority) of sorts.

When the organization operated with five or six domain teams, this approach ensured consistency and regulatory confidence. But as the platform scaled to more than twenty teams while simultaneously implementing ISO 20022 migration requirements and new cross-border payment flows, decision volume increased significantly.

Approval cycles extended from days to weeks. Teams began batching architectural changes to reduce review friction, unintentionally increasing the blast radius of each release. Platform teams were drawn into tactical debates rather than systemic risk management. The architecture function had not weakened; it had become the bottleneck.

The transition to a federated model redistributed decision rights to clearly defined domain teams operating within explicit architectural principles and automated guardrails. Policy checks were embedded into delivery pipelines. ADRs (Architectural Decision Records) were made transparent across domains. The central architecture group (TDA) shifted focus from approving changes to monitoring systemic risk and long-term coherence.

Within two quarters, release frequency increased, decision latency decreased, and regulatory

alignment improved, not because control was tightened, but because authority and accountability were aligned at the domain level.

The change was not about empowerment. It was about restoring throughput.

Redefining the Architect: From Controller to Constraint Designer

Decentralization does not eliminate the role of architects. It transforms it. In a centralized model, architects often act as reviewers and approvers. Their influence is direct but limited to the decisions they personally review.

In a decentralized model, architects shift from controllers to designers of constraints. Instead of approving individual implementation choices, they define the principles, boundaries, and feedback mechanisms that shape thousands of future decisions. This shift increases leverage. A controlling architect influences individual decisions. A constraint designer influences the decision-making system.

Architects articulate non-negotiable principles aligned to strategy, identify and reduce systemic coupling, define clear domain boundaries, design guardrails embedded in delivery systems, and observe divergence as data rather than deviation. Their role becomes less about directing traffic and more about designing the intersection.

Decision Boundaries and Structural Safety

Autonomy without structure creates ambiguity. Ambiguity increases coordination overhead. Coordination overhead reduces throughput. Clear decision boundaries are the structural foundation of scalable decentralization.

Bounded contexts, domain modeling, and explicit ownership definitions reduce hidden coupling. They define where authority begins and ends. They constrain the blast radius of change. Boundaries enable safe local optimization. Without boundaries,

decentralization produces fragmentation and duplicated effort. With boundaries in place, decentralization produces parallel evolution.

The goal is not to eliminate variation. It is to constrain its impact. Boundaries are the scaffolding that makes autonomy safe.

Federated Governance: Coherence Without Bottlenecks

Decentralization is often mistaken for the removal of governance. In practice, it requires stronger governance, but of a different kind. Federated governance centralizes what must remain coherent and decentralises what must remain adaptive.

Engineering strategy, security posture, regulatory standards, data governance, and shared platform capabilities require alignment across domains. Domain-level implementation detail, tooling choices, and service design require local adaptability. Architectural Decision Records preserve intent and reduce historical amnesia. Reversible experiments encourage learning before commitment. Guardrails embedded in CI/CD pipelines enforce standards structurally rather than through manual review.

When governance is embedded in systems rather than meetings, decision latency decreases while coherence remains intact.

The goal is not freedom from architecture. The goal is freedom *through* architecture.

Why Does This Matter Now

Organizations today are scaling under conditions very different from a decade ago. Regulatory complexity is increasing, digital products are evolving faster, and platform ecosystems are expanding across cloud-native and distributed architectures. At the same time, competitive pressure demands shorter release cycles and faster adaptation to market signals. The volume and velocity of architectural decisions are increasing

faster than traditional governance models can absorb.

Layer onto this the acceleration introduced by AI-assisted development and platform automation. The marginal cost of change is falling, but the systemic consequences of poorly governed change are rising. In this environment, architectural decentralization is not a cultural experiment. It becomes a structural imperative. Organizations that fail to distribute decision-making effectively will find that growth itself becomes the source of instability.

AI as the Intelligent CoPilot: The Satnav for the Roundabout

If decentralization removes the traffic officer and puts teams in the driver's seat, AI becomes the advanced satnav and driverassist system built into the dashboard. It does not take the wheel or assume responsibility for the journey. Still, it acts as a highleverage companion that increases situational awareness, reduces cognitive load, and strengthens the organization's ability to navigate complexity at scale.

In systems-thinking terms, AI expands the organization's perceptual bandwidth. Just as modern cars warn you when you drift into an occupied lane, AI surfaces blind spots in architectural reasoning. It interrupts reinforcing loops that would otherwise push teams toward unsafe divergence, local optimization, or repeated mistakes. It becomes a balancing feedback mechanism that stabilizes the system without re-centralizing control.

By analyzing the organization's "driving history" (its Architecture Decision Records, documentation, and patterns of past change) AI acts as a blackbox recorder. It can say, in effect, "The last three teams that took this exit crashed," transforming historical data into a learning loop. This strengthens the organization's adaptive capacity by making

institutional memory available at the moment of decision, not buried in a wiki.

Instead of expecting teams to memorize the entire highway code, the wiki, the compliance rules, and the architectural principles, Retrieval-Augmented Generation becomes a heads-up display. It projects the relevant constraints and guardrails onto the windshield at exactly the right moment. This reduces cognitive load and ensures that local decisions remain aligned with global principles. Governance becomes a realtime constraint system rather than a static document library.

AI can also analyze the map of the system to identify inefficiencies. It spots where coupling creates unnecessary detours, where dependencies slow flow, and where the architecture is taking the long way around. It highlights structural bottlenecks and proposes alternative routes that reduce latency and improve throughput. In systems terms, AI exposes structural leverage points, places where small changes produce disproportionately large improvements in flow.

The paved road, the platform, is transformed as well. Developers can interact with infrastructure using natural language, like issuing voice commands in a smart car. "I need a devbox" becomes the correct infrastructureascode, provisioned automatically, with pipelines scanned for potholes and vulnerabilities before the journey begins. This shifts platform interaction from manual configuration to intentdriven orchestration, reducing friction and increasing decision throughput without increasing risk.

In a busy roundabout, signaling is essential. AI-generated transcripts and summaries ensure that the intent of one driver is clearly understood by others, reducing the need for synchronous coordination. This lowers communication coupling, allowing teams to move independently without losing alignment. The system becomes more

loosely coupled, more resilient, and more capable of parallel evolution.

Before taking a risky turn on the real road, AI allows teams to run simulations. It can validate data contracts, test guardrails, and explore architectural options virtually. This creates a safe-to-fail environment, enabling experimentation without jeopardizing system stability. The organization gains the ability to probe the system, sense the impact, and adjust, which is a hallmark of adaptive, highmaturity sociotechnical systems.

But even the most sophisticated satnav is useless if the map is wrong.

AI models depend on clear decision domains, marked lanes, aligned incentives, fuel, unambiguous principles, and the highway code. Without these foundations, AI becomes a hallucinating passenger shouting random directions. Ambiguity in the system produces ambiguity in the agent. If the property lines of the system are not defined, AI cannot help you navigate them.

And while AI can suggest the route, it cannot pay the insurance excess. Transparency is non-negotiable, and accountability must remain firmly with the human driver. AI can extend autonomy, but it cannot absorb the consequences of poor decisions. AI-assisted systems, therefore require observability, auditability, and clearly defined decision domains to prevent silent drift at scale. The human should remain the final balancing loop in the system (at least for now).

The Sociotechnical Foundation of Scale

You cannot decentralize systems without decentralising authority. Structural changes must be accompanied by behavioral evolution. Teams require autonomy supported by context, skill, and psychological safety. They must also bear accountability, not just for delivery but also for longterm system health and operational resilience.

Trust becomes structural rather than personal. Leaders must trust teams to operate within defined guardrails. Teams must trust that decisions made within those guardrails will not be arbitrarily reversed. Autonomy without accountability creates chaos. Accountability without autonomy creates bureaucracy. Sustainable decentralization requires both. Scale emerges when responsibility and authority move together.

The Hidden Toll Booth: Finance as a Structural Constraint

Returning briefly to the roundabout: you remove the traffic officer, you redesign the intersection, flow improves, then movement stalls again. Why? Because finance remains centralized.

Architecture and finance are coupled systems. Financial governance shapes incentive structures, risk tolerance, and feedback loops. If it remains centralized, decision authority will eventually re-centralise with it. Architecture defines how technology can evolve; finance, on the other hand, defines how it is permitted to evolve. Annual project cycles, centralized budget approvals, costcentre accounting, and CapEx/OpEx optimization were designed to enhance predictability and cost control. They were not designed for adaptive, domain-driven growth.

When decision rights are decentralized, but funding authority remains centralized, autonomy becomes symbolic. Teams identify necessary improvements but lack the economic authority to act. Platform capabilities are treated as overhead rather than strategic investment. Shared services are underfunded because their value spans multiple cost centers.

Decision latency rises. Latency kills autonomy. You cannot run machinespeed autonomy on approval-speed funding.

Moving from project funding to capability funding realigns economic authority with architectural

ownership. Long-lived product teams, funded against value streams, create structures that support decentralized evolution. Platforms become investments in systemic resilience rather than discretionary costs. Architecture is not only technical design. It is operating-model design.

Data: The First Fault Line

In decentralized systems, data is often the earliest indicator of stress. Services can diverge within bounded contexts. Data crosses those contexts. Divergent definitions of core entities, unclear system-of-record ownership, and inconsistent telemetry introduce systemic risk long before visible failures appear.

Data demands stronger alignment than application logic. Clear stewardship, federated governance, shared semantic definitions, and transparent data contracts are essential for scalable autonomy. You can decentralize service design. You cannot decentralize accountability for shared data integrity.

Common Failure Modes

Fragmentation emerges when principles weaken and hidden coupling increases. Tooling diverges, patterns multiply, and cognitive load rises. Duplicated effort follows unclear ownership. Reinvention feels faster than reuse but increases systemic complexity over time. Strategic drift occurs when local optimization overwhelms enterprise coherence. Platform teams can become accidental gatekeepers when guardrails are ambiguous, and enforcement becomes manual, reintroducing central bottlenecks.

These are not failures of decentralization itself. They are symptoms of incomplete decentralization. Clarity creates flow. Ambiguity creates friction. A hybrid model that centralizes shared services, security standards, and data governance while federating domain-level decisions provides structural balance.

Real-World Patterns That Sustain Autonomy

Scalable decentralization is not improvised; it is institutionalized. Clear interaction models between teams reduce coordination friction. Documented architectural decisions preserve context and reduce repeated debate. Explicit principles create shared mental models. Golden paths reduce cognitive load. Communities of practice sustain alignment across domains.

These patterns transform decentralization from aspiration into repeatable capability. They make autonomy durable.

Architecting Autonomy as a Leadership Discipline

The traffic officer at the junction eventually reaches capacity. The roundabout flows because it distributes decision-making within clear rules. The lesson is not about roads. It is about growth.

Decentralizing architectural decision-making increases organizational concurrency. It reduces latency, distributes expertise, and enables parallel domain evolution. But it demands coherence across technical design, governance, finance, data, and culture.

Autonomy is not the absence of architecture. It is the outcome of intentional architectural design. Organizations that master this shift build adaptive, resilient, high-throughput systems. They grow without collapsing under their own complexity. They evolve without centralizing every decision.

Those that do not will oscillate between bureaucracy and chaos. Your intersection will eventually reach capacity. The question is not whether to redesign it. The question is whether you will design for scale deliberately or wait for congestion to force the change.

In high-growth environments, architectural decentralization is not optional. It is the structural precondition for sustainable scale.



Decentralizing Architecture at Scale: A Lean Value Tree Approach to Autonomous Teams, Federated Governance, and Platform Engineering

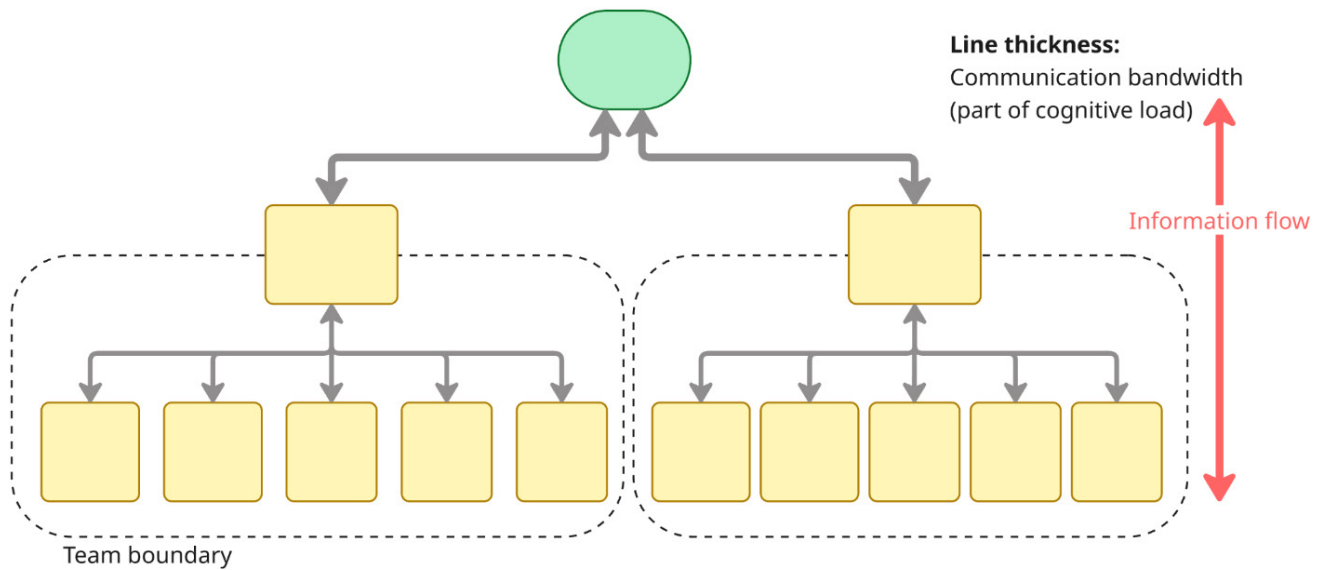
István Hegedűs, Principal Engineer
Mahdi Pourziaei, Staff Engineer & Technical Lead
Ignacio Goyetche, Software Architect
Maher Gamal, Software Architect

Introduction

In the early stages of an organization, architectural alignment is a byproduct of the proximity of the organization's members. This is recognized as [Conway's Law](#). The people who work on the system are often involved in many stages of its development and share a context organically. Thus, "the right way" to build is often understood intuitively. [Team Cognitive Load](#) and communication needs fit within the bandwidth of

the people involved, as it flows either up or down the organization chart.

As organizations grow to include more teams with their own roadmaps, alignment becomes less organic and requires more active maintenance. The intuitive next step often becomes centralized architectural control, where a few senior members tightly oversee the work of various teams to ensure quality.



However, as the organization continues to grow, this creates a bottleneck and efficiency issues. High-stakes review boards and rigid gatekeeping strain communication bandwidth, forcing a lose-lose trade-off: slow delivery or bypass governance, accruing significant architectural debt.

Information becomes trapped in organizational “crevices”—the private Slack channels of tenured engineers or the undocumented mental models of a few key architects. For the teams building product value, the symptoms become immediate and painful:

- Teams spend time preparing “defense decks” to bring review boards up to speed for a decision.
- The need for rework becomes the norm as teams build in isolation due to a lack of shared “living context”.
- A shadow architecture of “quick fixes” becomes the brittle ecosystem that requires each new project to work around.

Leaders must pivot from **gatekeepers** to building a decentralized, **contextual architecture**. This enables teams to make and drive decisions that deliver

business outcomes quickly, without breaking the world around them.

Mapping the Path to Decentralization

It is best to see this ambitious vision as an organizational transformation on a gradient of change rather than a monolithic leap.

The transformation is built on three core pillars:

- **Architecture:** defining clear boundaries, durable contracts, and federated governance.
- **Organization:** aligning team structures and clarifying decision rights.
- **Engineering Culture:** promoting shared norms and engineering platforms that make the “right” choice the “easy” choice, ensuring autonomy remains responsible rather than chaotic.

In this article, we use a [Lean Value Tree](#) to map a strategy that achieves the vision using key concepts from the industry standard, [Team Topologies](#). This strategy is a reflection of our own lessons learned going through this change, the roundabout way, and we have aimed to support its parts using references to case studies or documented experience of well-known companies.



The Lean Value Tree

The Lean Value Tree (LVT) serves as our map for the [Value Stream](#), ensuring that every architectural decision and organizational bet is directly tied to improving [Flow](#). With this alignment, our Goals and Bets become more than just abstract ideas. They are specific interventions to improve how work gets done, delivering an “idea” to “production”.

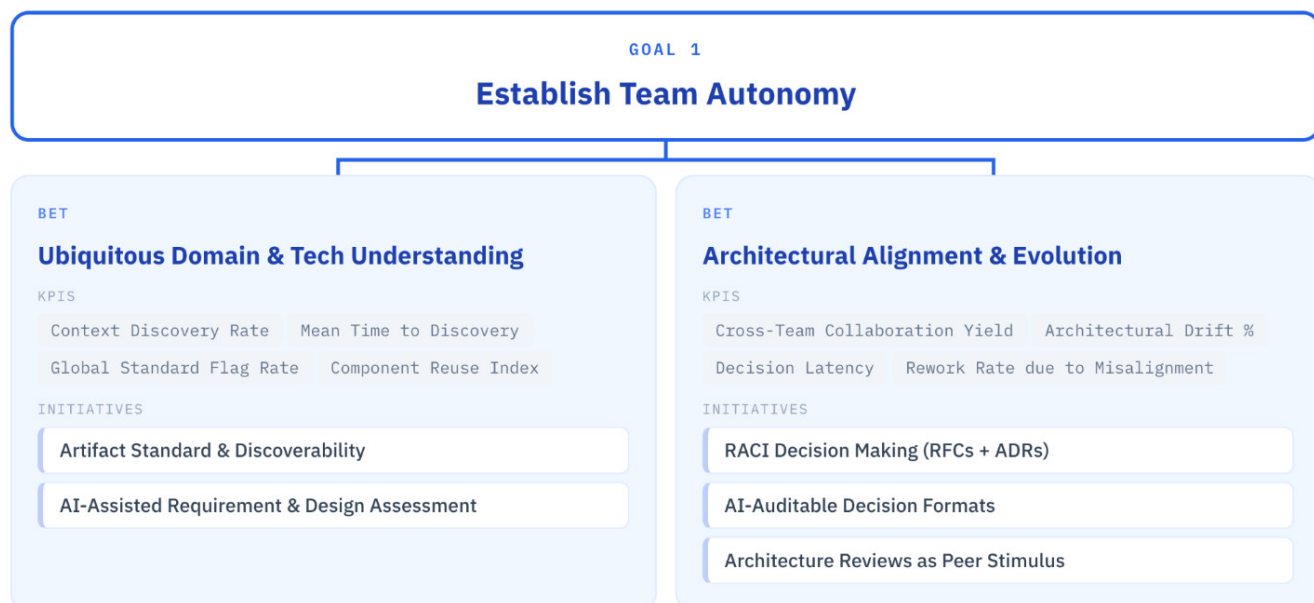
Goal 1: Establish Team Autonomy

To achieve true autonomy, organizations must navigate the tension between speed and stability. As [DORA research highlights](#), high-performing organizations achieve both by implementing

“Guardrails, Not Gates” (more on this in Goals 2 and 3). This approach shifts governance from a manual, bottleneck-prone process to a “context architecture” that empowers teams to make high-quality decisions at the edge without constant oversight.

Bet: Ubiquitous domain & tech understanding

Hypothesis: [Stream-aligned Teams](#) often fail when they lack visibility into the broader ecosystem, leading to duplication and “rumor-driven development”. Ensuring that the “context” of the organization is as accessible as the code itself will broaden visibility and cross-pollination, leading



to self-autonomous teams able to function with minimal blocking dependencies.

KPIs to evaluate the hypothesis by

- **Context Discovery Rate:** how often team members successfully navigate and utilize other teams' artifacts
- **Mean Time to Discovery:** the overall time spent searching for internal documentation or ownership details
- **Global Standard Flag Rate:** how often AI flags requirements and solution designs for conflicting with cross-organization guidelines
- **Component Reuse Index:** the frequency of reuse for common libraries and components across the org

Initiative: Artifact standard & discoverability

To ensure teams can locate the right information without social gatekeeping, domain and technical knowledge must be standardized in discoverable catalogs. This is foundational for a [Team API](#). Spotify addressed this challenge by building [Backstage](#), which utilizes a rigorous "Descriptor Format" for catalog entities. By standardizing metadata for every software component—whether a service, library, or website—they ensure that ownership and documentation are discoverable by default. This allows teams to self-serve the context they need, effectively treating internal documentation with the same rigor as shipping code.

Initiative: AI-assisted requirement and solution design assessment

As organizations scale, the volume of local decisions quickly outpaces the capacity for manual human review. While Amazon's ["Two-Pizza Teams"](#) model enables speed, it generates architectural change that overwhelms traditional governance. To maintain decentralization without chaos, organizations can deploy AI agents as a first line of defense. By ingesting system context, these

agents validate local requirements against global standards, providing architectural oversight at scale without the bureaucratic drag of manual reviews.

Bet: Architectural alignment & evolution

Hypothesis: While autonomy allows for local speed, the goal is to align teams on critical non-negotiables while leaving implementation details open.

KPIs to evaluate the hypothesis by

- **Cross-Team Collaboration Yield:** measuring how cross-team interactions have changed over time with respect to the volume of successfully integrated RFCs and ADRs.
- **Architectural Drift Percentage:** calculating the variance between the deployed codebase and documented ADRs, segmented by the criticality of priority components.
- **Decision Latency:** The average time from an RFC (Request for Comments) being opened to an ADR (Architectural Decision Record) being finalized (measures the speed of the "Advice Process").
- **Rework Rate due to Misalignment:** The percentage of PRs/Stories that require architectural changes after a "Peer Stimulus" review has occurred.

Initiative: Embracing RACI in Decision Making

To prevent autonomous teams from operating in silos, organizations must establish clear boundaries and stakeholder relationships, often using a RACI (Responsible, Accountable, Consulted, Informed) framework for the Architecture Advice Process. Following [Harmel-Law's](#) advice process, an engineer proposes a solution via an RFC to consult experts and impacted teams, then documents the final decision as an immutable ADR alongside the code. This process democratizes decision-making while ensuring all relevant stakeholders are engaged.

Initiative: AI-Auditable Architecture Decision Formats

The explosion of AI coding tools means code is generated orders of magnitude faster than it can be reviewed. As systems evolve rapidly, static ADRs can quickly lose touch with reality, leading to architectural drift where daily habits outpace target designs. To combat this, organizations are adopting AI-auditable decision formats. By using LLMs and automated fitness functions, AI agents can continuously compare the current codebase implementation against documented ADRs to flag divergence. For instance, [NN Group](#) built a multi-agent system where generating agents and validating agents work together to fetch information and ensure ADRs reflect the current architectural context.

Initiative: Architecture reviews as peer stimulus

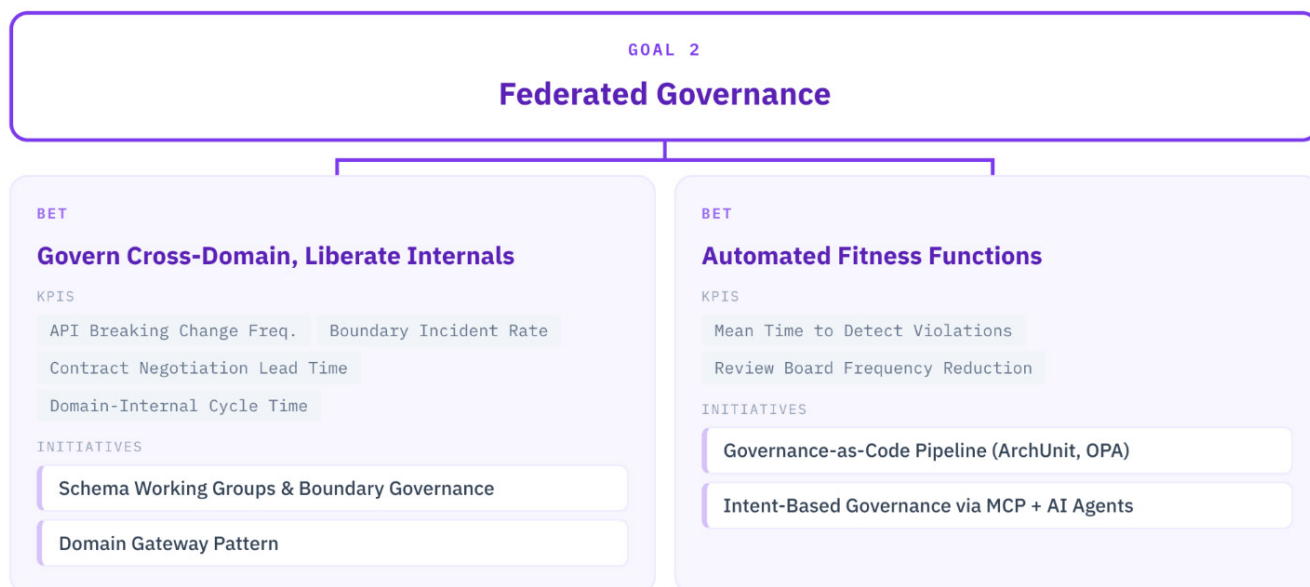
Autonomy requires accountability, not isolation. Instead of treating architecture reviews as a bureaucratic hurdle, they can serve as a forcing function for quality and education. These sessions allow teams to present their designs for cross-team scrutiny, with a focus on “bar-raising” standards such as security, scalability, and operational excellence. Forums like [Open GI’s Architectural](#)

[Advisory Forum](#) act as spaces for cross-team visibility and peer feedback rather than approval gates. This ensures that while a team fully owns its roadmap, it remains accountable for the organization’s collective architectural health.

Goal 2: Coherence Through Federated Governance

Autonomy without coherence leads to a mess of incompatible systems, duplicated infrastructure, and divergent standards. Spotify experienced exactly this when their fragmented developer tooling ecosystem led to what [they called “rumour-driven development”](#).

The instinct is to recentralize control, but that simply trades fragmentation for bottlenecks. The alternative is to federate decision-making, centralizing decisions that must be consistent (e.g., cross-domain boundaries, interoperability standards, security baselines) while liberating teams to make all other decisions within their domains. This is what makes autonomy sustainable at scale, federated governance is not a counterweight to autonomy but its enabler. Teams *can only be truly autonomous* when the boundaries and interfaces around them are well-governed.



The governing question is *where* to focus governance effort and *how* to enforce it without reintroducing bottlenecks.

Bet: Govern Cross-Domain interactions, liberate domain internals

Hypothesis: If architects govern cross-domain interactions (API contracts, event schemas, integration patterns) while leaving domain-internal decisions to teams, the organization achieves coherence at the boundaries and autonomy within domains.

KPIs to evaluate the hypothesis by

- **API Change Frequency**, rate of unplanned breaking changes to cross-domain contracts.
- **Boundary Incident Rate**, number of production incidents traceable to cross-domain integration failures vs. domain-internal failures.
- **Contract Negotiation Lead Time**: The time spent in the “Consulted” phase of the RACI for cross-domain changes (measures the efficiency of the federated boundary).
- **Domain-Internal Cycle Time**: Mean time from first commit to production for changes that do not impact cross-domain contracts (measures true “liberation”).

Initiative: Invest Governance Effort at the Boundaries

Not all architectural decisions carry equal governance weight. Internal domain decisions are high-frequency, low-blast-radius choices that teams are best positioned to make. Cross-domain interactions are lower-frequency but high-blast-radius, a breaking change can ripple across multiple teams. Federated governance should concentrate effort where coupling costs are highest: *at the boundaries*. This maps to [Khononov’s coupling model](#), where contract coupling is the sensible default between domains: “anything internal to the subsystem behind its interface can be changed more confidently and rapidly. As long as the

contract is not broken, the change is safe”. The governing architect’s job is to keep cross-domain relationships at contract coupling, preventing degradation into stronger forms. In the AI era, this becomes even more critical: coding agents can rapidly rebuild domain internals, but the contracts between domains are durable architectural investments.

Netflix illustrates boundary-focused governance at scale. Its [federated GraphQL architecture](#) (250+ Domain Graph Services, 60+ teams) is governed through a Schema Working Group that ensures cross-domain consistency. Netflix later reinforced this with [UDA \(Unified Data Architecture\)](#), which lets teams model their domains independently while connecting them through explicit mappings — making the boundaries between domains visible, discoverable, and governable.

Bet: Automated Fitness functions

Hypothesis: Encoding architectural standards as automated fitness functions and policy-as-code will result in faster feedback loops, reduced governance bottlenecks, and improved compliance across autonomous teams. This directly improves Flow efficiency.

KPIs to evaluate the hypothesis by

- **Mean Time to Detect Architectural Violations**, this should improve dramatically.
- **Architecture Review Board Frequency**, how often review boards need to convene should be reduced and also the agenda should shift toward cross-cutting concerns.

Initiative: Governance through Automated Fitness functions

This reframes architectural governance from periodic manual reviews to continuous, automated validation, transforming the role of the Architect from an approver to an exception handler. A mature tooling ecosystem already supports this — from

ArchUnit for structural rules to Open Policy Agent for policy-as-code.

Yet traditional fitness functions share a fundamental limitation: they are brittle when scaled beyond individual teams, because they couple tightly to specific implementation details.

Ford and Richards argue that agentic AI and the Model Context Protocol (MCP) solve this brittleness problem. Their insight is that enterprise architects have always struggled to express governance concerns like “validate referential integrity across our distributed data stores” without knowing each project’s specific databases, schemas, and deployment topology, and when teams change those details, the fitness function breaks, even though the architectural concern hasn’t changed. A general integration layer can be defined in MCP servers for agents: the enterprise architect states governance intent in natural language, an MCP server at the project level exposes implementation-specific fitness functions as tools, and an AI agent mediates between the two. When the project evolves — say, adding a third database — the internal tooling changes, but the architect’s

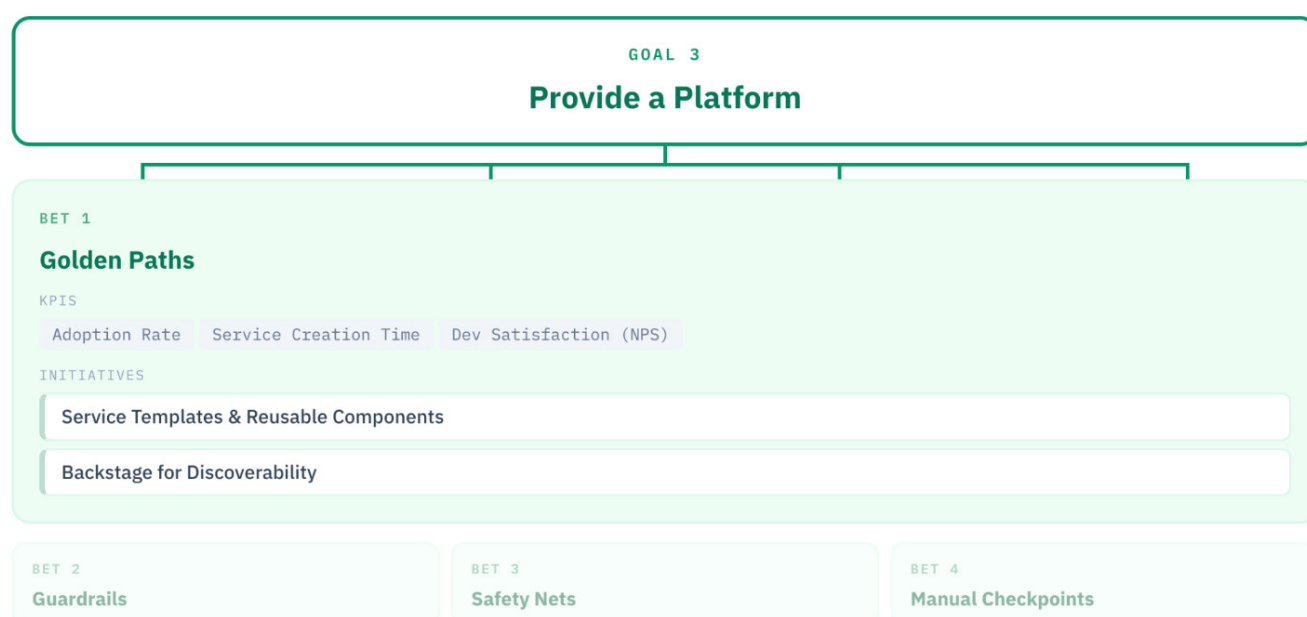
governance request stays the same. This represents a shift from implementation-coupled governance to intent-based governance, enabling fitness functions to finally scale from team-level to enterprise-level without brittleness.

As Ford advises for Fitness functions adoption: “You don’t have to eat the entire elephant”. Start by identifying a single, high-pain governance gap, the rule most often violated or the review that creates the worst bottleneck — automate it as a fitness function, and measure the before/after impact.

Goal 3: Provide a Platform

The **Platform’s goal is to enable the autonomous teams of our future decentralized organization to operate within accepted patterns and in compliance with the governance model by providing a platform that makes the right choices easy** and the wrong choices hard.

The Platform goal supports the two other pillars (**Autonomy** and **Federated Governance**) by embedding standards directly into the developer experience. Rather than relying on documentation people don’t read or reviews that slow teams down,



Your turn – define KPIs, initiatives & culture changes using the same Bet format +

the platform encodes alignment into the tools teams already use.

[Google Cloud's taxonomy of platform engineering control mechanisms](#) provides a useful framework here:

1. **Golden Paths:** Steer teams toward the right choice (proactive, low friction)
2. **Guardrails:** Prevent catastrophic choices automatically (preventive, non-negotiable)
3. **Safety Nets:** Detect failures and enable fast recovery (reactive, resilient)
4. **Manual Checkpoints:** Inject human judgment where it matters most (selective, high-stakes)

These four layers map naturally to four bets under the Platform goal of the LVT. In this section, we cover only the “Golden Paths” bet and show how the many platform engineering concepts can be brought together to support the LVT.

Bet 1: Golden Paths

Hypothesis: Creating “golden paths” (i.e., opinionated, preconfigured, officially supported workflows) will improve compliance and reduce deviations from desired patterns while preserving the agility gained from team autonomy. The right choice becomes the easy choice.

KPIs to evaluate the hypothesis by

- **Adoption rate:** % of teams using golden path templates vs. custom solutions
- **Service Creation Time:** Mean time required to go from a service concept to a live, monitored, and compliant production instance (golden path vs. non-golden-path)
- **Dev. Sat. (NPS):** Developer Satisfaction score on ease of running a new service

More options are available for measuring the success of this bet. Louron Pratt covers this in his article on [internal product management](#).

Initiative: Install Golden Paths for Teams to Follow

Spotify has made extensive use of [Golden Paths](#) in the same way and more to resolve fragmentation across autonomous teams.

Artifacts of the initiative:

Artifacts are the tangible resources you can declare as deliverables for this initiative. These are what the Platform’s users use directly to facilitate their workflow.

Some examples include:

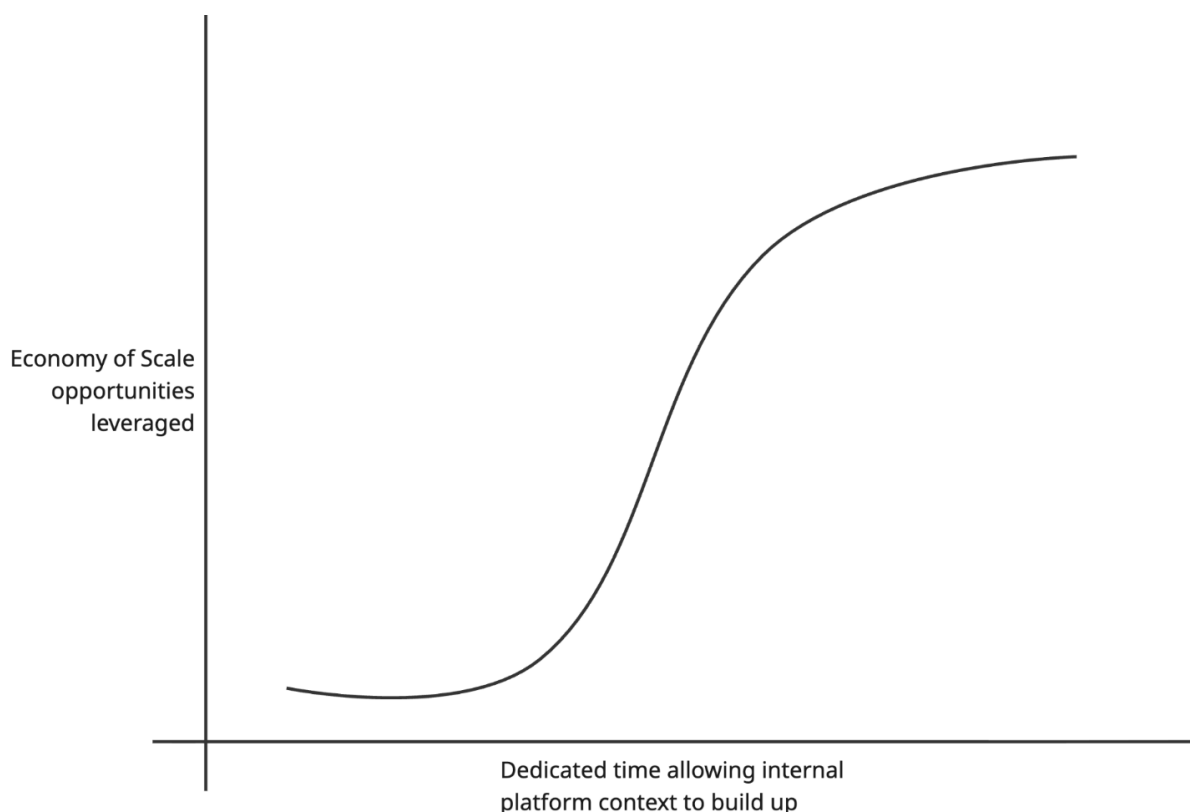
- Reusable components for logging, monitoring, auth, and application parameters.
- Service templates for the service types your organization creates. e.g., for Lambda functions, for long-running workloads, etc.
- Custom Terraform modules
- Backstage for discoverability of systems, APIs, data, and even Golden Path tutorials.

There are many resources that provide clever solutions for platform artifacts (ex: [microservices.io](#), [dzone.com](#)).

People & Culture changes for the initiative:

Recognizing platform work as ‘real work’ and staffing a Platform Team in exchange for efficiency and control of decentralization chaos are key aspects of top-down support.

In smaller organizations, a dedicated team of SREs or Ops/Infra Engineers may not be available. Instead, proficient individuals can fill these roles by acting as Enabling Teams and dedicating X% of their time to the Platform Team’s roadmap. However, it’s important to note that this investment



follows the Sigmoid Curve, as the Platform Team must shape its learning and product development to achieve “product-market fit,” which requires collaboration and time within a stable team.

Thus, adopting a culture of “*Platform as a Product*” is a key enabler of success for this initiative to ensure the Platform is designed to minimize the cognitive load of its customers—the Stream-aligned Teams using a [Team-first Approach](#).

Some examples of concrete process changes that directly support this culture are:

1. New Platform Team roadmap items must link directly to measurable developer outcomes.

Culture proof: Platform Team roadmap reviews look to KPIs to decide what to prioritize.

2. Developer feedback is collected through structured, documented surveys.

Culture proof: Roadmap adjustments are explicitly tied to feedback data.

3. If “*Support Deflection*” is a KPI, retros include “support themes” as a standing agenda item to trigger creation of roadmap items.

Culture proof: Every recurring support request becomes a signal for a Platform feature request.

How the Four Bets Work Together To Deliver the Goal

You can reuse the Bet format above for the other 3 layers (ex: Bet 2 is based on Guardrails)—each with their own initiatives, artifacts, culture, and process changes—to cover the spectrum of decisions teams need to make:

Most decisions are handled by the lowest-friction layer, and only the highest-stakes decisions escalate to human review. This is what makes decentralization sustainable. In essence, the platform handles alignment by default, so people

only need to intervene where judgment genuinely matters.

The Path Forward

Instead of thinking about decentralization as a destination to reach, think of it as capabilities to unlock one “Initiative” at a time towards decentralization wherever effort is focused.

To determine your focus area, identify your organization’s most acute symptom:

- **Is delivery stalled through tight approval processes?** Start with **Goal 1 (Autonomy)** and place a bet.

- **Is your ecosystem struggling with difficulty in bridging services or data?** Focus on **Goal 2 (Federated Governance)**.
- **Is architectural compliance seen as “extra work” that slows teams down?** Prioritize **Goal 3 (The Platform)**.

By stabilizing context, boundaries, and tooling, we enable scale across many fronts simultaneously. This improves innovation and opens opportunities for decentralizing and improving access to other paradigms (ex: Data Mesh).

Start small, prove value in short cycles, and iterate to mature the organization into a self-regulating ecosystem.



Decentralized Architecture Governance: From Review Boards to Golden Paths and Practitioner-Led Standards

Charles de Courval, Principal Software Architect

Raj Pejathaya, Software Architect

Robbie dela Victoria, Staff Software Architect & Senior Manager of Platform Engineering

Elena Rico, Senior Software Engineer & Technical Lead

Carl Tompkins, Software Architect

The Trellis and the Town Square

A decade ago, the DevOps revolution promised teams the autonomy to build and manage their own domains. This successfully dismantled traditional delivery bottlenecks, but many organizations soon discovered a hidden cost: freedom without shared direction leads to fragmentation. Without a cohesive framework, companies find themselves battling “Frankenstein” architectures—a disjointed patchwork of redundant security layers, clashing networking solutions, and incompatible databases.

The Paradox of Autonomy

Imagine a vine growing without a trellis. Early growth is impressive, but eventually, branches tangle and compete for resources. In software, this manifests as teams re-solving problems others have already cracked—one team adopting a managed streaming service while another self-hosts a message broker. This “technical sprawl” creates an operational burden that eventually exceeds the cost of the original centralized bottleneck.

This is the paradox of autonomy: giving teams total freedom often leads to a cognitive load tax that slows them down more than the old gatekeepers ever did.

Redefining the Architect: From Ivory Tower to Town Square

The greatest obstacle to architectural agility is the perception of architects in the Ivory Tower—a centralized group of senior leads who hand down diagrams and dictate to delivery teams they rarely interact with. In a decentralized ecosystem, this model is not just inefficient; it is a primary driver of technical friction. When an architect acts as a gatekeeper, they become a single point of failure. To scale, the profession must undergo a fundamental shift from being the primary source of technical decisions to becoming the steward of the decision-making environment.

The Architect as an Environmental Steward

In the “Town Square” model, the architect’s value is no longer measured by the brilliance of their individual designs but by the velocity of the teams they support and the coherence of the system they contribute to. This is the shift from “Command and Control” to “Context and Constraints.” As a steward, the architect focuses on the scalable levers of the organization:

- **Engineering the Constraints:** Instead of reviewing every database schema, the steward defines the global non-negotiable, security baselines, data sovereignty rules, and interoperability standards. These are the “rules of the road” that allow teams to drive as fast as they want without risking a multi-car pileup.
- **Curating the Enablers:** The architect partners with platform teams to ensure that the Internal Developer Platform (IDP) provides the right abstractions. They identify recurring patterns across the organization and “productize” them into reusable primitives, ensuring that no team has to solve the same infrastructure problem twice.

- **Managing the Feedback Loops:** Stewardship involves monitoring the health of the technical ecosystem. By analyzing reliability signals, incident reports, and developer sentiment, the architect identifies where standards are failing and where new “Golden Paths” need to be cleared.

The Town Square Principle: Discoverability over Dictate

If the Ivory Tower was defined by closed-door sessions, the “Town Square” is defined by radical transparency. The objective is to make architectural reasoning **discoverable**. In a high-performing organization, a developer shouldn’t need to book a meeting with an architect to understand why a specific technology was chosen or how to integrate with a legacy system.

This transparency is achieved through three primary mechanisms:

1. **Architecture Decision Records (ADRs):** By capturing the “why” behind a decision, the trade-offs considered, and the context at the time, we prevent the “architectural archeology” that slows down future teams.
2. **The Technology Radar:** This acts as the organization’s GPS, clearly signaling which technologies are encouraged (Adopt), which are being piloted (Trial), and which are being sunset (Deprecate).
3. **Advice Forums:** Replacing formal review boards with open design sessions where any engineer can present a proposal for peer feedback. This turns architecture into a collaborative “social” activity, fostering a culture where the best ideas win regardless of title.

By moving into the Town Square, the architect stops being a distant regulator and starts being a high-leverage consultant. They move from the business of *making* decisions to the business of *improving*

decisions, ensuring that the organization's technical intelligence is distributed, rather than centralized.

Frameworks for Decentralized Decision-Making: The Town Square Model

The most common failure point in decentralization is the vacuum left when a central Architecture Review Board (ARB) is dissolved without a replacement. To scale safely, organizations must transition to an Architectural Guild, a practitioner-led body composed of representatives embedded within delivery teams. Unlike a traditional ARB, which often acts as an external auditor, the Guild functions as a collaborative forum where standards are co-authored by the people responsible for implementing them. This ensures that governance is grounded in the reality of the codebase rather than ivory-tower theory.

Defining Boundaries through the Blast Radius

The core mechanism of a decentralized framework is a clear heuristic for delegation. In a healthy Town Square, authority is not a binary switch but a spectrum determined by a decision's blast radius and its reversibility. By categorizing decisions into impact levels, we can automate the "Default to Local" principle:

- **Level 1: Local Context (High Reversibility):** Decisions that are confined to a single service or domain—such as internal refactoring, library updates, or localized data schemas—remain entirely within the delivery team. These are "two-way doors" that can be easily undone if they prove suboptimal.
- **Level 2: Cross-Domain (Moderate Reversibility):** Decisions that impact shared interfaces, API contracts, or neighboring domains require a "Consult and Notify" approach. The proposing team retains the decision rights but is obligated to surface the design in the Town Square via a lightweight RFC (Request for Comments).
- **Level 3: Enterprise Structural (Low Reversibility):** These are the "one-way doors"—

choices like switching a global identity provider, moving from a relational to a document-based core data model, or altering the organization's security posture. These high-stakes shifts require formal consensus from the Federated Architecture Council to ensure the long-term integrity of the entire ecosystem.

The Guardrail Heuristic

To prevent the Guild from becoming a bottleneck, we must establish clear "Guardrail Tests" that trigger escalation only when necessary. A decision is escalated to the broader Council only if it threatens systemic safety (violating compliance or security baselines), introduces tight coupling between unrelated domains, or incurs a prohibitive "reversal cost."

From Command to Discovery

In this "Town Square" model, the architect's primary tool is the Architecture Decision Record (ADR). The goal is to move from "Checking for Permission" to "Discovering Reasoning." When teams document their trade-offs and context in a discoverable way, the organization builds a living repository of technical intelligence. This transparency creates a self-correcting system; as teams see how their peers solved similar challenges, a natural selection of best practices occurs. The Guild's role then becomes one of harvesting these local innovations and elevating them into global standards, ensuring the organization learns as fast as it grows.

The Internal Developer Platform: The Engine of Autonomy

If decentralized architecture provides the trellis that guides growth, the Internal Developer Platform (IDP) is the ecosystem that sustains it. An IDP is far more than a collection of infrastructure tools or a curated list of cloud services; it is a unified, self-service layer designed to abstract away the cognitive tax of modern distributed systems. By moving away from "ticket-ops" and manual hand-offs, the IDP empowers delivery teams to own their entire lifecycle—from the initial commit to production

monitoring—without requiring them to become infrastructure experts.

The Solution: Architecture as a Trellis

Decentralizing architecture requires a fundamental cultural shift. We must move away from the “**Ivory Tower**” model, where architects act as distant gatekeepers, and toward a **Supportive Stewardship** model, where architects focus on designing the environment in which teams operate.

The goal is to provide a **Trellis**: a structure that guides growth and provides stability without dictating the shape of every individual branch. This means defining the critical constraints that matter at the system level—such as security, interoperability, and global data standards—while leaving context-heavy, day-to-day decisions to the delivery teams who are closest to the code.

By replacing manual approval gates with the **Town Square** principle of radical transparency and automated guardrails, we align accountability with the parties making the decisions. This ensures that the “right” choice is not just a mandate, but the path of least resistance for every engineer.

Treating the Platform as a Product

The primary reason most internal platforms fail is that they are built as static projects rather than evolving products. [1](#) [2](#) To truly enable autonomy, a platform must be governed by product management principles, led by a Technical Product Manager (TPM) who treats developers as customers. This shift in mindset is critical; instead of forcing tools on teams, the platform team must engage in continuous user discovery to identify where developers are getting “stuck”.

The success of such a platform is not measured by compliance mandates, but by the “pull” it creates—how quickly it becomes the path of least resistance for developers. This move from Platform Push (enforced tooling) to Platform Pull (organic adoption) is the fundamental differentiator of a

high-performing IDP. A compelling platform earns adoption because it makes life easier, effectively eliminating the temptation for “Shadow IT” workarounds that inevitably occur when centralized systems are too rigid or complex.

Engineering the Golden Path

The most potent architectural lever within the IDP is the concept of the golden path. This is a pre-architected, highly opinionated workflow that comes with security defaults, logging, observability, and CI/CD pipelines pre-integrated. When a developer follows this path, they are not just using a template; they are opting into a state of implicit governance. They are automatically compliant with the organization’s architectural standards because those standards are encoded directly into the platform’s APIs.

However, true autonomy also implies the freedom to stray. A team whose domain requires a specialized graph database or a non-standard language should have the right to pursue a “silver path”. The trade-off is the operational tax: by moving off the golden path, the team accepts full responsibility for the total lifecycle of their custom choice, including patching and maintenance. When the cost of divergence is clear, and the reward for alignment is high, the “path of least resistance” naturally trends toward architectural coherence.

From Permission Culture to Guardrail Culture

The final objective of the IDP is to replace a “Permission Culture” with a “Guardrail Culture”. In traditional organizations, the architect is a gatekeeper who must review every infrastructure request. In a mature Trellis, the architect works with the platform team to encode those reviews into automated policy-as-code.

We measure the success of this transition through both quantitative and qualitative lenses, looking for improvements in DORA metrics—specifically a decrease in “Lead Time for Changes” as manual approvals vanish. Simultaneously, we track “Time

to Value”—how quickly a new engineer can make their first production deployment. When a junior developer can ship code on day one without filing a single ticket, the platform has successfully turned architectural theory into operational reality.

The Cultural and Leadership Evolution: From Command to Context

The transition to a decentralized architecture is often mistaken for a purely technical endeavor, yet the most sophisticated Internal Developer Platform (IDP) will fail if the underlying culture remains rooted in centralized command and control. Shifting to a “Town Square” model requires a fundamental evolution in how leadership is exercised and how teams perceive their accountability.

The Foundation of Psychological Safety

True architectural autonomy cannot exist without psychological safety. When we delegate authority to the “edges,” we are asking teams to take on significant risk. If the organizational response to a failed technical experiment is punitive, teams will naturally retreat into the safety of centralized approval boards, effectively re-instating the very bottlenecks we sought to remove.

To avoid this, leadership must foster an environment where technical debt and design failures are treated as systemic feedback. This means institutionalizing blameless post-mortems—not just for operational outages, but for architectural decisions. When a decentralized choice leads to an integration conflict or a scalability issue, the investigation should focus on why our “Town Square” mechanisms or “Guardrails” failed to surface the risk early, rather than seeking a team to blame. By rewarding transparency over perfection, we encourage teams to openly document their reasoning in ADRs, turning every local failure into a global learning opportunity.

The Shift to Strategic Intent

In the Ivory Tower, architects are the “Chief Decision Makers.” In the Town Square, they become “Context Providers.” This follows the military principle of

Commander’s Intent: providing a clear, high-level “North Star” vision while granting teams the freedom of maneuver to decide how to reach it.

For an architect, this means moving away from dictate-driven governance and toward intent-driven stewardship. Instead of mandating a specific database, leadership defines the required outcomes—such as five-nines availability, GDPR compliance, and sub-second latency. Once these “North Star” parameters are set, the delivery teams are empowered to navigate the technical landscape to meet them. This shift requires a high degree of trust, but it is the only way to scale architectural thinking across hundreds of engineers without the architect becoming a single point of failure.

Incentivizing the Right Behaviors

Finally, a sustainable autonomous culture must align its incentive structures with its architectural goals. Traditionally, engineering cultures disproportionately reward “building from scratch”—the hero who creates a new system in a vacuum. To sustain a decentralized ecosystem, the organization must equally value those who contribute to the trellis.

In our ‘Town Square’ model, engineers find satisfaction in being **active citizens** who contribute to the IDP and mentor their peers. This participation creates a culture of ownership where the Trellis is shaped by the very people it supports. While we encourage this creative contribution, we remain disciplined by applying a predictable operational tax to any divergence that creates unnecessary complexity. Autonomy is not a license for anarchy; it is a trade-off. A team is free to choose a custom silver path outside of organizational standards, but they must accept the responsibility for the total lifecycle of that choice, including patching, security, and maintenance. When the cost of divergence is clear, and the reward for alignment is high, the path of least resistance naturally trends toward architectural coherence.

In Summary

Decentralization works best when it is supported by clear decision boundaries, practitioner-led governance, an enabling platform, and a culture that treats failure as feedback rather than individual blame. But knowing what to build is different than knowing where to start. For leaders and architects navigating this transition, following a phased adoption path can create the most value with the least disruption.

The Architecture of Autonomy: The Trellis and the Town Square

To move away from the bottlenecks of a centralized “Ivory Tower,” we must provide both the structure to grow and the light to see by. This is achieved through the dual pillars of the **Trellis** and the **Town Square**.

The **Trellis** represents our technical foundation—the Internal Developer Platform and its “Golden Paths.” Much like a physical frame, it doesn’t dictate exactly where every leaf should sprout, but it provides the essential support and automated guardrails that

allow teams to climb higher and faster without creating a tangled, unmaintainable mess. By encoding standards directly into the tools engineers use every day, the Trellis ensures that the “right thing” is always the “easy thing.”

However, a structure is only as strong as the community that tends to it. The **Town Square** represents the shift from “checking for permission” to “discovering reasoning.” By moving architectural discourse into an open, transparent forum—powered by discoverable ADRs—we ensure that intelligence is distributed rather than hoarded. In this model, engineers aren’t just passive users of a platform; they are active citizens who find deep satisfaction and ownership in contributing back to the Trellis and mentoring their peers. Together, these concepts transform architecture from a static gatekeeping function into a living, high-velocity ecosystem built for sustainable speed. When we stop trying to control every branch of the vine and instead focus on building the strongest possible trellis, we create an environment where creativity and autonomy don’t just survive: they thrive.



InfoQ | CERTIFIED ARCHITECT PROGRAM

5 weeks online cohort.
Real software architecture challenges. Immediate ROI.

Start your 5-week learning journey today.



Decentralized Architecture Governance: Decision Rights, Enabling Platforms, and the Shift from Gatekeeper to Adviser

A N M Bazlur Rahman, Java Champion & Software Engineer
Leslie Rock Dupervil, Cloud & Security Architect
John Haman, Software Architect

Introduction

In software development organizations, a handful of architects can feasibly serve as central gatekeepers, deciding on frameworks, vetting designs, and ensuring coherence. However, as the development scales, this model may crumble. New teams continually push forward, the technology landscape shifts, and a central review board quickly becomes a bottleneck. In a [recent article](#), Olatunji highlights reasons why most enterprises hire architects, establish governance frameworks, and only later

ask how architecture should actually operate. By the time the question is asked, business units have already developed their own patterns and shadow architectures, forcing the central team to catch up and creating friction. The result is a slow, top-down process that cannot keep pace with modern product teams.

On the other hand, decentralizing architecture does not mean anarchy. A healthy marketplace is not a free-for-all; it is an ecosystem of vendors

operating under shared norms and common infrastructure. This article shows leaders how to transition from centralized control to a network of empowered teams without sacrificing coherence, security, or compliance. It synthesizes lessons from research, real-world case studies, and practitioner experiences to provide a concrete roadmap.

Why Centralized Fails at Scale

Centralized models have real strengths: consistent standards, strong security, and regulatory compliance. They make sense when compliance is critical, a nascent company is still executing on a technical founder's early vision, or technology sprawl is out of control.

But the hidden costs are significant. Architectural decisions can take far longer than they should, and adopting new technology can stretch out for well over a year. Frustrated by delays, business units build their own solutions and bypass governance entirely.

These problems run deeper than slow approvals. When architecture functions as a gate rather than a guide, architects spend most of their time in meetings and on compliance checks rather than on shaping direction. Without a clear operating model, decision rights blur, architectural choices get relitigated and overridden within months. Over time, the organization accumulates "governance debt": informal workarounds and political arrangements that take years to undo.

Sociotechnical Benefits of Decentralized Architecture

Mitigating the challenges associated with centralized architectures will increase speed and reduce cost; however, there are important sociotechnical benefits that are less tangible.

The friction often experienced by teams in centralized architectures is reduced by empowering individual teams. The friction shifts from operational to intellectual, moving from a

procedural checklist to a tool that challenges existing methodologies and architectural concepts among peers through a communication loop. This beneficial, positive debate naturally leads to broad leaning and supports a learning-driven culture over a command-and-control culture.

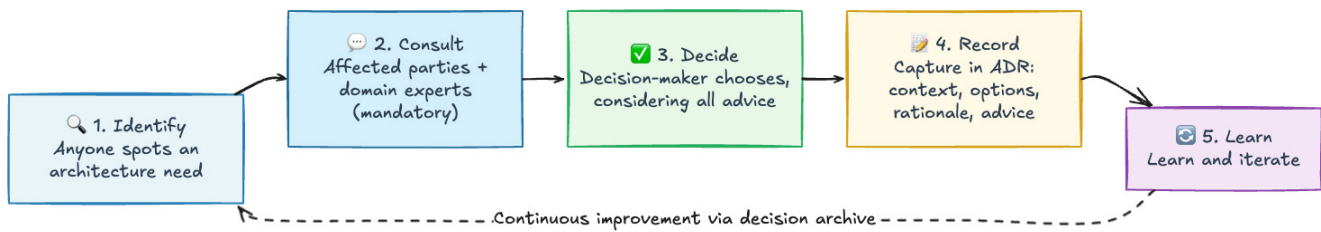
A learning-driven culture further reduces the burden of managing an engineering organization by enabling all individuals to continuously upskill themselves. As knowledge increases, the talent pool within the organization grows, providing a robust supply of talent already onboarded and ready to advance to meet the needs of a successful organization.

Redefining the Architect's Role

Decentralization is as much a cultural shift as a structural one. In *Facilitating Software Architecture* (O'Reilly, 2024), Andrew Harmel-Law describes trying both traditional approaches, the ivory tower (review boards, sign-offs, dedicated architecture functions) and the hands-on alternative (moving from team to team, trying to do all the required design in all the places it was needed). His conclusion: neither works well. At best, even when his architectural work was good, he slowed teams down by becoming a bottleneck. At worst, it was a losing battle; teams knew their code, their domain, their pipelines, and their customers far better than any single architect ever could. His response was to stop dictating decisions and start facilitating them. He proposed the advice process.

The Advice Process

Harmel-Law's Architecture Advice Process is straightforward: anyone can make an architectural decision, provided they first consult those who will be affected and those with relevant expertise. This decentralizes decision-making while preserving alignment through consultation and trust. Underpinning it is a social contract; decision-makers must genuinely seek and consider advice, and colleagues must trust them to act on it responsibly.



The process has five steps:

1. **Identify the need.** Anyone on a team can recognize an architectural need, a technology choice, an API boundary, or a data model change. No gatekeeping is required; recognizing the need is enough to begin.
2. **Consult.** The person who identified the need talks to two groups: those affected by the decision (downstream teams, API consumers, operations staff) and people with expertise in the relevant area. Architects are explicitly part of this second group. But their role extends beyond advising when asked; they start conversations, maintain the supporting elements (advisory forum, principles, tech radar), and remain decision-takers for cross-team concerns. What changes is that they lose the monopoly on deciding and the power to veto. Consultation is mandatory, but it is input, not approval.
3. **Decide.** The decision maker weighs all advice and makes the call. Anyone can fill this role; there is no requirement that the same person who identified the need be the one to do so. In practice, team-level decisions are typically taken by teams or team leads, while those in cross-team roles decide cross-cutting concerns. What matters is that the ADR makes the decision maker explicit, creating clear accountability. The decision taker is not bound to follow the majority view; they are bound to have genuinely considered it.
4. **Record.** The decision is captured in an [Architectural Decision Record](#) (ADR), a lightweight, version-controlled document that records the context, the options evaluated, the rationale, and the advice received. ADRs create a searchable archive that prevents relitigating settled choices and helps new team members understand prior reasoning without reopening old debates.
5. **Learn.** The decision archive feeds back into future decisions. Teams iterate on both the architecture and the process itself. This feedback loop is what makes the advice process evolutionary rather than merely permissive.

Three principles anchor the whole system:

1. **Anyone can decide.** Decision-making authority is not reserved for architects or review boards. Architects participate as advisers, not approvers. This does not mean decisions are unconstrained; platform guardrails, fitness functions, and policy-as-code still enforce organizational boundaries automatically. The distinction is between a human veto, which the Advice Process removes, and systemic constraints, which remain.
2. **Consultation is mandatory.** Both affected parties and relevant domain experts must be sought before a decision is taken. Advice is input, not permission; no individual holds a veto, but skipping consultation violates the social contract.
3. **Decisions must be recorded.** ADRs create the institutional memory that turns individual decisions into organizational learning.

Two practical challenges surface at scale: finding the right advisers, and ensuring consultation is genuine rather than performative.

Locating advisers is a discovery problem, not a process problem. Harmel-Law's primary mechanism is the Architecture Advisory Forum (AAF), a weekly open-invite session where decision-makers present draft ADRs and affected parties self-identify. This inverts the burden; instead of the decision-maker mapping every stakeholder in advance, the forum makes decisions visible and lets expertise surface organically. For known dependencies, service catalogs such as Backstage provide a second layer. Ownership metadata and dependency graphs answer "who owns what my decision touches" and "what else might break" without requiring the decision-maker to know the organizational map already. The combination of a catalog for structural dependencies and a forum for human expertise scales better than expecting any individual to trace the full blast radius alone.

Genuine engagement is harder to mandate but easier to make visible. The ADR's Advice section requires recording who was consulted and what they said. Shallow consultation becomes apparent in the document trail. Harmel-Law frames this as "explicit ADR accountability is a check to the reckless." The social contract also binds advisers: those who decline to engage when consulted forfeit standing to relitigate later. When contributors see their input visibly shape an outcome, engagement becomes self-sustaining.

Platforms as Enablers

Even with the organizational approach described above, decentralized architectures, by definition, place new responsibilities on each development team. One way to reduce the cognitive burden on teams is to provide streamlined infrastructure tooling aligned with core architectural principles. The net impact is that teams move faster with less friction in an aligned architectural direction. The vehicle for this approach is Platform Teams.

As described in *Team Topologies (Pais and Skelton)*, Platform teams facilitate other teams in various ways. One such mechanism for decreasing

cognitive burden is providing Internal Developer Platforms, which address this by offering self-service APIs, tools, documentation, and paved paths that encode best practices into reusable components, so developers don't need to reinvent deployment pipelines, networking, authentication, or monitoring.

A well-designed platform includes self-service infrastructure, modular components, golden paths, API-first design, strong observability, and built-in security. Paula Kennedy, COO and co-founder of Syntasso, [has described how platform engineering addresses the growing cognitive load on development teams by offering curated paths and self-service tooling that reduce the burden of navigating complex infrastructure choices.](#) By managing shared concerns like compliance and governance, platform teams enable stream-aligned teams to focus on delivering business value. Kennedy also cautions that this approach risks shifting cognitive load onto platform teams rather than eliminating it, making investment in platform team experience equally important.

Critically, platforms must be treated as products. Product managers should gather user feedback, prioritize features, and track adoption. Without this product mindset, platforms risk becoming the very bottleneck they were designed to eliminate.

Federated Governance & Guardrails

Autonomy without alignment is chaos. Federated governance strikes a balance by splitting responsibilities: a central team defines security, compliance, and interoperability outcomes rather than prescriptive tools and techniques. Furthermore, the platform teams provide shared services, while domain teams own their data and services. It is not pure decentralization; it combines central standards with local execution, using contracts to enforce consistency at the boundaries.

Guardrails are how governance becomes operational. Rather than requiring human approval

at every step, modern practices favor automated enforcement: fitness functions that continuously test adherence to architectural principles (e.g., latency thresholds, security policies) and abstract data contracts which focus on outcomes rather than explicit definitions. Outcomes are well-defined and measurable by individual teams through automated tests and quality metrics, enforced by CI/CD pipelines and updated via the advice process.

Data mesh, as articulated by Zhamak Dehghani, exemplifies these concepts within the realm of data: each domain treats its data as a product, with responsibility for quality, discoverability, and the data lifecycle. Organizations that have adopted domain-oriented data ownership report improvements in latency and data quality, though results vary with context and implementation maturity.

AI Agents and Adaptive Governance

AI is reshaping enterprise architecture. Gartner predicts that by 2026, 40% of enterprise applications will integrate task-specific AI agents, up from less than 5% in 2025. These agents are evolving from assistive tools into execution engines, orchestrating workflows, performing CRUD operations, and managing transactions via protocols such as Anthropic's Model Context Protocol (MCP), an open standard for connecting AI models to external data sources and tools. As agents take on more execution, the back end's role shifts: from business logic to governance, permissioning, and cost control.

For decentralized architectures, this creates two imperatives. First, as teams embed AI agents into their services, governance must ensure that those agents comply with policy, respect data privacy, and avoid runaway costs. Second, AI can reinforce guardrails at scale: machine learning models can classify sensitive fields, detect anomalies, and recommend remediation. Integrating AI into the platform's observability and policy engine

enables organizations to scale governance without increasing headcount.

Common Pitfalls and How to Avoid Them

Decentralizing architecture unlocks speed and autonomy, but it also exposes organizational fault lines. Organizations might face several recurring challenges when shifting from centralized control to federated, domain-oriented architecture.

Pitfall 1: The Architect's Identity Crisis

Organizational decentralization often aligns with modern delivery models that emphasize team autonomy and domain ownership, as discussed in *Team Topologies*. However, while distributed decision-making increases responsiveness and ownership, it frequently exposes a less visible structural issue: the architectural role becomes ambiguous within a federated model.

As organizations decentralize, the role of the architect inevitably transforms. Architects accustomed to centralized governance and formal approval authority often struggle to redefine their influence within federated, domain-aligned teams. Without a centralized gatekeeper, architectural authority shifts from control to guidance. This transition creates tension: architects must allow autonomous teams to make decisions—even imperfect ones—while maintaining responsibility for long-term coherence, risk management, and systemic integrity. Such tension is not a failure of decentralization; it is a predictable phase in the maturation of a federated architecture model.

How to avoid it:

Redefine the architect's identity early. Move from "approver" to "advisor," reinforcing responsibilities such as stewarding architectural principles, curating shared patterns, mentoring subject-matter experts, and supporting teams through the advice process. Transparency around expectations helps architects transition from authority figures to enablers.

Pitfall 2: Early-Stage Turbulence Without Guardrails

In the early stages of decentralization, organizations commonly experience a phase of predictable turbulence: architectural approaches begin to diverge, decision-making accelerates, and ambiguity emerges around who holds authoritative expertise. While teams gain speed, decisions are not always informed by the appropriate systemic perspective.

To stabilize this phase without reverting to centralized control, mature organizations introduce lightweight but intentional governance mechanisms. Clear subject-matter expertise is formally recognized rather than informally assumed. Federated architectural forums are established at both enterprise and domain levels, creating structured spaces for architects and senior engineers to review decisions, share Architectural Decision Records (ADRs), and maintain cross-domain alignment.

This approach reflects the facilitative leadership model described in ***Facilitating Software Architecture***, where architectural effectiveness emerges not from gatekeeping authority but from enabling structured dialogue, shared understanding, and collaborative decision-making.

How to avoid it:

Do not confuse decentralization with the absence of structure. Establish a governance nucleus early:

- identify SMEs to support the advice process,
- define cross-sector architecture forums,
- standardize documentation via ADRs, and
- create a single place where architectural knowledge accumulates. This keeps decision-making fast without drifting into fragmentation.

Pitfall 3: The Speed vs. Capability Tension

One of the sharpest tensions in decentralized environments emerges between delivery speed and technical capability. As decision-making authority shifts to domain teams, expectations for rapid

cloud adoption and service deployment increase. However, many teams lack deep expertise in networking, identity management, infrastructure automation, security hardening, and cost governance.

The consequence is predictable: local optimizations, fragile implementations, duplicated effort, or repeated escalations to specialized central experts. Autonomy exists in theory, but dependency persists in practice.

This dynamic reveals a critical insight:

Decentralization without enablement creates hidden centralization.

When teams are granted ownership without the necessary technical foundations, decision cycles slow, risks increase, and informal bottlenecks reappear.

How to avoid it:

Sustainable decentralization requires investing in enablement before expanding autonomy. The most effective mechanism is a deliberate platform-engineering capability that abstracts complexity and embeds governance within well-defined delivery paths.

By providing standardized environments, reusable infrastructure templates, automated guardrails, and curated self-service capabilities, organizations shift governance from approval processes to engineered constraints. In this model, compliance and architectural integrity are built into the platform itself rather than enforced through review boards.

As described in *Team Topologies*, platform teams exist precisely to reduce cognitive load and enable stream-aligned teams to move quickly without compromising systemic integrity.

Conclusion

Decentralizing architecture is neither a technical refactoring nor an ideological leap. It is an organizational transformation that demands careful design of decision rights, investment in enabling platforms, cultural change toward trust and transparency, and, increasingly, the integration of AI into governance.

The journey cannot be rushed. Each organization must assess its regulatory environment, maturity and ambitions. Decentralization without enablement is worse than centralization; but decentralization with the right guardrails, platforms and social

contracts unleashes speed, learning and resilience that no central function can match.

By adopting a deliberate operating model, empowering domain teams through the advice process, building self-service platforms, enforcing guardrails through policy-as-code and embracing AI, leaders can cultivate a thriving marketplace of teams. In this model, architects become gardeners who set boundaries and nurture growth rather than clerics who guard the gates. The payoff is faster delivery, happier teams, greater resilience and an architecture that can evolve as quickly as the business it serves.

CONTRIBUTORS



Steven O'Kennedy

I am a Director of Architecture at Liberty IT with over 20 years' experience in the software industry. I have worked across multiple sectors, including pharmaceuticals, public service, health, and insurance, and have held roles spanning consulting, project management, innovation, software engineering, technical architecture, and enterprise architecture. I am passionate about staying current with industry trends, particularly as we move deeper into the age of AI.



Andreas Hörsken

I'm a Lead Software Engineer at Verlag C.H.Beck, where I build software with cross-functional teams. I focus on architectural methods and processes that enable sustainable, resilient systems—especially API-centric solutions. I'm particularly interested in how AI is changing the way we learn, collaborate, and make architecture decisions, for better and for worse.



Wejdan Bagias

Wejdan Bagias is a Senior Machine Learning Manager specializing in enterprise-scale AI/ML systems, agentic AI, and multi-agent architectures. She leads applied machine learning initiatives focused on production-grade AI systems, data pipelines, and scalable model delivery. Wejdan works closely with cross-functional teams to translate complex business problems into robust, maintainable AI solutions, with a strong emphasis on quality, automation, and MLOps best practices.



Pushpdeep Mishra

I bring a strong combination of Leadership, Technical expertise, Managerial capability, and Community engagement skills. I specialize towards reducing Redundancy, Toil and increasing efficiency for a given task. I value strong work ethics and enjoy fostering a culture of continuous learning and improvement within my team. I lead national-level initiatives in the fields of EdTech, innovation, and healthcare. I am passionate about exploring emerging technologies, leadership practices, software and hardware product development.



Shweta Aggarwal

is a Senior Solution Architect at Liberty IT with 20+ years of experience designing and delivering scalable, resilient, cloud-native applications and distributed data/analytics platforms. She has led architecture across both greenfield and brownfield initiatives, with a focus on modernizing legacy ecosystems while enabling reliable, high-performance systems at scale. A technology enthusiast with a strong interest in emerging capabilities and constraints, she enjoys applying pragmatic innovation to solve complex business problems.



Ron Klein

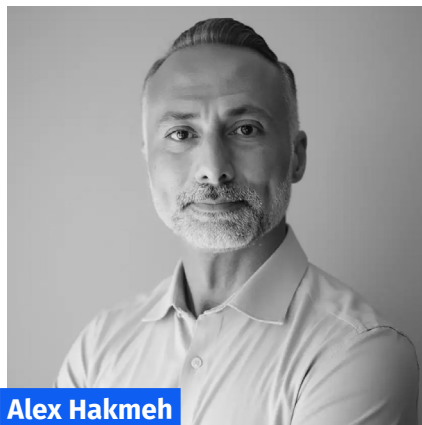
is a senior software architect at Elementor, leading large-scale R&D initiatives and driving technical standardization across teams. In his spare time, he contributes to the Node.js open-source ecosystem, teaches advanced software engineering courses, and mentors engineers on career growth. Outside of tech, he enjoys films, music, and podcasts.


Ruth Montgomery

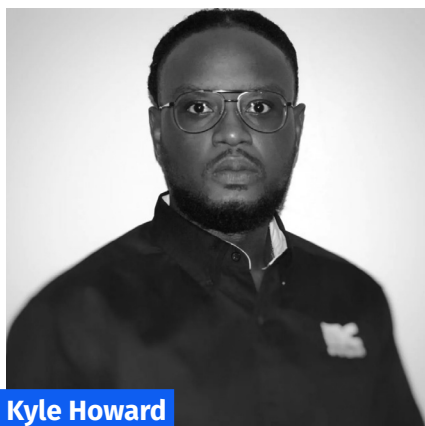
is a Senior Solutions Architect at Liberty IT in Belfast. Her career has evolved from building enterprise integration platforms to designing architectures for treasury and financial systems. She specializes in creating systems that support team autonomy while maintaining coherence across domains, particularly the intersection of technical design and team structures.


Ajeesh George

is a Solutions Architect with over 19 years of experience delivering enterprise technology solutions across the e-commerce and retail sectors. He specialises in cloud-native architecture and API-led integration, with extensive expertise in engineering event-driven microservices and establishing modern technology standards for complex systems.


Alex Hakmeh

is a senior technology and delivery leader with experience scaling complex systems across fintech, payments, and regulated enterprise environments. He has led large transformation initiatives focused on distributed systems, architectural governance, and operating model design. His work centres on enabling autonomous teams while maintaining system coherence, with a particular interest in decision boundaries, federated governance, and the relationship between architecture and organisational scale.


Kyle Howard

I am a Senior Backend Engineer mostly working in fintech environments. I have an interest in architecting solutions and finding innovative ways to remove roadblocks.


Christian Johansen

is an independent software architect and founder of GalaxiTech, with a master's in mechanical engineering from DTU. His career spans 30 years — from software for car simulators, flight simulators, rat respirators, and cucumber glass fillers to domain-driven design, sociotechnical architecture, and event-driven systems. He organizes the Copenhagen DDD meetup and currently works with AI, Event Modeling, CloudEvents, EventCatalog, and many other things.


Dana Katzenelson

is a Staff Backend Software Engineer at Panther Labs based in Silicon Valley. She has over a decade of experience with a focus on product engineering across multiple industries.



Brian Rhoten

is the Solution Architect of Victim Services for Public Safety at Equifax, focusing on high-availability notification systems. His work centers on real-time data integration and building resilient platforms for community safety initiatives. He aims to bridge the gap between complex data engineering and the delivery of life-saving information to millions of users. Outside of work, Brian is an avid traveler and a steadfast goblin wrangler.



Warren Gray

is a technology leader focused on building durable, coherent platforms in high-change environments. For over a decade at Arctic Wolf, he has led the architectural evolution of the XDR platform through hypergrowth and acquisition, aligning autonomous teams around highly evolvable systems that scale cleanly and deliver customer value without excess complexity. He holds a degree in Computer Science and Software Engineering from the University of Waterloo.



Charles de Courval

As a Lead Principal Software Architect at SimCorp, I specialize in steering complex technical roadmaps and transitioning legacy platforms toward modern, micro-service architectures. With over 15 years of experience leading high-performing Agile teams, I bridge the gap between business objectives and technical excellence through robust CI/CD pipelines and cloud-native engineering.



Raj Pejathaya

brings 16 years of experience in HealthCare industry with main expertise on Provider platforms. He helped build a greenfield hospital for Cleveland Clinic in the middle east. He has worked with world class clinicians and learnt from the best to practice compassionate care through technology practices. He worked on Epic and Cerner EHR, clinical application and their integration and interoperability. He helped setup Information As A Service discipline to expose data for predictive and operational support.



Robbie dela Victoria

is a Staff Software Architect and Senior Manager of Platform Engineering at AbbVie, where he leads teams that build developer platforms and cloud infrastructure to make software delivery faster, safer, and more reliable. With a background spanning architecture, distributed systems, and engineering leadership, he focuses on practical platform capabilities, including paved roads, self-service tooling, observability, and sensible governance, to help organizations scale without slowing teams down.



Elena Rico

Elena Rico is a Senior Software Engineer and Technical Lead at Toast based in Cary, North Carolina. She brings extensive experience designing and delivering highly scalable, microservices-based systems using Java, Spring, and RESTful APIs. With a background spanning fintech, telecommunications, and aviation software, she is known for leading complex initiatives under tight deadlines while maintaining high standards of quality and performance.



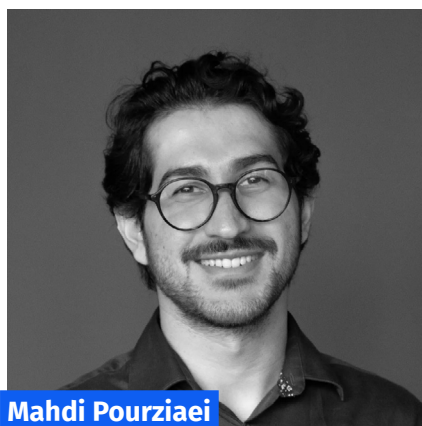
Carl Tompkins

is a Software Architect with over ten years of experience designing and modernizing distributed systems, with a focus on cost optimization and scalable infrastructure. His areas of interest include serverless computing and data architecture.



István Hegedűs

is a Principal Engineer at Kobo where he leads the modernization of search platform. With over a decade of experience as a Solution Architect across EPAM Systems and Valtech, he specializes in large-scale e-commerce transformations. István holds multiple composable commerce certifications in Commercetools and Contentful, and is a certified IASA Architect (CITA-F).



Mahdi Pourziaei

is a breadth-first Staff Engineer and Technical Lead at Fashion Cloud with over a decade of experience in software development and architecture. He focuses on evolving fragmented products into cohesive platform ecosystems, most recently driving Fashion Cloud's multi-year consolidation of a multi-product suite of applications following mergers and acquisitions.



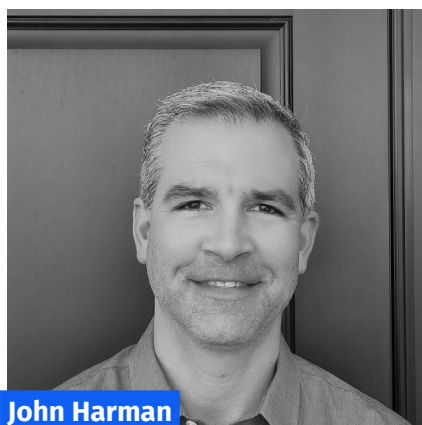
Ignacio Goyetche

is a software architect specializing in scalable distributed systems and enterprise platform design. With almost 15 years of experience delivering high-impact solutions in corporate environments, he is passionate about aligning technology decisions with business outcomes and helping teams build resilient, maintainable platforms. His interests include architecture modernization, reliability, scalability, and engineering excellence.



Maher Gamal

19+ years in the trenches of crafting web-based solutions for corporates and startups, took over from architectural envisioning all the way up to production engineering. Nominated as one of Software Community Experts among Orange Labs Worldwide. Pioneered at stretching technological increments on various granularities. Led and coached different team sizes, being central to teams raising their technical bar effectively trying to bring out the best in everyone.



John Harman

John Harman is a scientist turned software product manager turned software producer who manages Harman-Solutions LLC, a biopharmaceutical focused consultancy. John currently hold a patent pending on the Fragmented Persistence Object Services model for data persistence and is actively driving the new era of software production.



Leslie Rock Dupervil

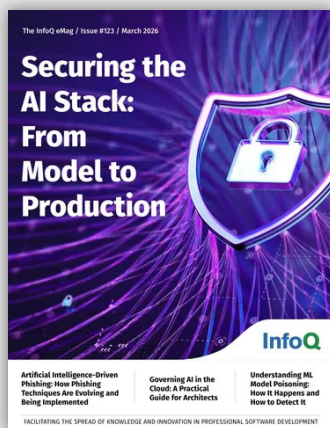
is a cloud and security architect with more than 18 years of experience designing, modernizing, and securing enterprise-scale systems. He specializes in Azure architecture, cloud security, identity governance, and Infrastructure as Code, with hands-on expertise guiding organizations through complex transformations. A Certified Cloud Security Professional (CCSP), his work spans large-scale cloud migrations, Zero Trust implementations, data-exchange platforms, and governance frameworks in highly regulated environments.



A N M Bazlur Rahman

is a Senior Staff Software Engineer at Hammerspace with over a decade of experience in Java and related technologies. His contributions to the ecosystem have been recognized with the title of Java Champion. He is the founder and moderator of the Bangladesh Java User Group, where he has organized developer meetups and educational events since 2013. Bazlur is also a regular speaker at international conferences and Java user groups, often presenting on modern Java concurrency, virtual threads, and the evolution of the Java platform.

Curious about previous issues?



AI has officially shifted from experimentation to production, outpacing legacy defenses and creating a volatile new security landscape. This challenge is defined by three critical frontiers: data poisoning, AI-driven phishing, and shadow cloud governance.



This eMag is designed to bring together insights from industry practitioners to explore how architecture can evolve in the era of AI. From rethinking platforms, automation practices and embracing decentralized decision-making to redefining the architect's role as coach and enabler, the contributions here highlight strategies for navigating technical and organizational challenges.



This special edition of The InfoQ eMag, contains a comprehensive collection of our popular InfoQ Trends Reports from 2025, a year with both evolution and revolution within the landscapes of technology, software development trends. This collection does not just reflect the past year's technological trends. We aspire to use it as a guide for future exploration and innovation.